BrainAge Connect

Complete Code Listing

Line-numbered backend, support tools, tests and browser-console sources.

Student	Module	Assessment
Boyu Yuan B01089181	COM661 Full Stack Strategies and Development	CW1 - Back-end API

This listing prints the shipped human-authored/AI-assisted code files in full. Long source lines wrap visually; continuation lines do not receive a new source line number. Secrets, runtime files, downloaded tools and real research data are excluded.

Prepared 28 October 2026, 01:46 (UTC+08:00)

Application: infant MRI metadata and user-supplied precomputed result management. No MRI inference or clinical diagnosis is claimed.

Contents

Complete Code Listing 1

Contents 2

1. Source inventory 4

2. app.py 6

3. brainage/__init__.py 7

4. brainage/admin.py 9

5. brainage/analytics.py 10

6. brainage/auth.py 12

7. brainage/common.py 14

8. brainage/samples.py 16

9. scripts/__init__.py 19

10. scripts/bootstrap.py 20

11. scripts/export_database.py 21

12. scripts/generate_openapi.py 22

13. scripts/import_database.py 25

14. scripts/import_dhcp.py 26

15. scripts/import_results.py 28

16. scripts/package_submission.py 29

17. scripts/run_postman.py 30

18. scripts/seed_demo.py 31

19. scripts/verify_exports.py 32

20. tests/conftest.py 33

21. tests/test_api.py 34

22. static/app.js 37

23. static/index.html 39

24. static/style.css 40

25. setup.ps1 41

26. start.ps1 42

27. stop.ps1 43

28. RUN_TESTS.cmd 44

29. START_DEMO.cmd 45

30. START_PROJECT.cmd 46

31. STOP_PROJECT.cmd 47

32. requirements.txt 48

33. requirements-dev.txt 49

34. .env.example 50

35. .gitignore 51

1. Source inventory

34 files; 1754 source lines. Full SHA-256 digests are included beside each file and in the accompanying manifest.

File	Lines
app.py	9
brainage/__init__.py	89
brainage/admin.py	49
brainage/analytics.py	109
brainage/auth.py	101
brainage/common.py	117
brainage/samples.py	215
scripts/__init__.py	0
scripts/bootstrap.py	25
scripts/export_database.py	28
scripts/generate_openapi.py	145
scripts/import_database.py	48
scripts/import_dhcp.py	129
scripts/import_results.py	34
scripts/package_submission.py	37
scripts/run_postman.py	52
scripts/seed_demo.py	52
scripts/verify_exports.py	33
tests/conftest.py	56
tests/test_api.py	210
static/app.js	58
static/index.html	12
static/style.css	1
setup.ps1	10
start.ps1	65
stop.ps1	28
RUN_TESTS.cmd	4
START_DEMO.cmd	4
START_PROJECT.cmd	4
STOP_PROJECT.cmd	4

File	Lines
requirements.txt	7
requirements-dev.txt	3
.env.example	5
.gitignore	11

Postman request/test scripts and saved examples are supplied in the original collection JSON and documented in the companion reference. Documentation files are provided separately, not substituted for executable source.

2. app.py

SHA-256: 95c0623e8a467e9ef4a3eab73fc1c792ca125e3c3a4e0feb5c28c5a54a2366e2

```
"""Entrypoint: python app.py; local-only Waitress server."""
import os
from brainage import create_app

app = create_app()

if __name__ == '__main__':
    from waitress import serve
    serve(app, host='127.0.0.1', port=int(os.getenv('PORT', '5000')))
```

3. brainage/__init__.py

SHA-256: 37688e15f28a893227574725614c090430e65b4237204ae068c6f8037a72ef47

```
"""Application factory. Database handles belong to the application, not global imports."""
import os
from pathlib import Path
from bson import ObjectId
from dotenv import load_dotenv
from flask import Flask, jsonify, send_from_directory
from flask_cors import CORS
from pymongo import MongoClient
from pymongo.errors import DuplicateKeyError, PyMongoError
from werkzeug.exceptions import HTTPException
from .common import APIError

ROOT = Path(__file__).resolve().parent.parent

def ensure_indexes(database):
    database.users.create_index('username', unique=True)
    database.samples.create_index('sample_code', unique=True)
    database.samples.create_index([('owner_id', 1), ('age_at_scan_weeks', 1)])
    database.samples.create_index([('visibility', 1), ('status', 1), ('sample_code', 1)])
    database.samples.create_index([('source', 1), ('age_at_scan_weeks', 1)])
    database.revoked_tokens.create_index('jti', unique=True)
    database.revoked_tokens.create_index('expires_at', expireAfterSeconds=0)
    database.audit_logs.create_index('created_at')

def create_app(config=None):
    load_dotenv(ROOT / '.env')
    app = Flask(__name__, static_folder=str(ROOT / 'static'))
    app.config.update(MONGO_URI=os.getenv('MONGO_URI', 'mongodb://127.0.0.1:27018'),
        MONGO_DB=os.getenv('MONGO_DB', 'brainage_connect'), JWT_SECRET=os.getenv('JWT_SECRET'),
        MAX_CONTENT_LENGTH=1024 * 1024, TOKEN_SECONDS=3600, TESTING=False)
    if config:
        app.config.update(config)
    if not app.config['JWT_SECRET'] or len(app.config['JWT_SECRET']) < 32 or
app.config['JWT_SECRET'].startswith('replace-'):
        raise RuntimeError('Set a random JWT_SECRET of at least 32 characters in .env.')
    client = MongoClient(app.config['MONGO_URI'], serverSelectionTimeoutMS=5000, tz_aware=True)
    app.extensions['mongo_client'] = client
    app.extensions['mongo_db'] = client[app.config['MONGO_DB']]
    client.admin.command('ping')
    ensure_indexes(app.extensions['mongo_db'])
    CORS(app, resources={r'/api/*': {'origins': os.getenv('CORS_ORIGINS',
'http://localhost:4200,http://127.0.0.1:4200').split(',')}})
    from .auth import bp as auth_bp
    from .samples import bp as samples_bp
    from .analytics import bp as analytics_bp
    from .admin import bp as admin_bp
    for bp in (auth_bp, samples_bp, analytics_bp, admin_bp):
        app.register_blueprint(bp, url_prefix='/api/v1')

    @app.get('/api/v1/health')
    def health():
        client.admin.command('ping')
        return {'status': 'ok', 'application': 'BrainAge Connect', 'version': '1.0.0'}

    @app.get('/')
    def home():
        return send_from_directory(app.static_folder, 'index.html')

    @app.get('/api/v1/openapi.json')
    def openapi():
        return send_from_directory(str(ROOT / 'docs'), 'openapi.json')

    @app.errorhandler(APIError)
    def api_error(exc):
        return jsonify(error={'status': exc.status, 'message': exc.message}), exc.status

    @app.errorhandler(DuplicateKeyError)
    def duplicate(exc):
        return jsonify(error={'status': 409, 'message': 'A record with that unique value already exists.'}), 409

    @app.errorhandler(HTTPException)
    def http_error(exc):
        return jsonify(error={'status': exc.code, 'message': exc.description}), exc.code

    @app.errorhandler(PyMongoError)
    def database_error(exc):
```

```
        app.logger.exception('Database operation failed')
        return jsonify(error={'status': 503, 'message': 'Database temporarily unavailable.'}), 503

    @app.errorhandler(Exception)
    def internal_error(exc):
        app.logger.exception('Unexpected request error')
        return jsonify(error={'status': 500, 'message': 'Internal server error.'}), 500

    @app.after_request
    def headers(result):
        result.headers['X-Content-Type-Options'] = 'nosniff'
        result.headers['X-Frame-Options'] = 'DENY'
        result.headers['Cache-Control'] = 'no-store'
        return result
    return app
```

4. brainage/admin.py

SHA-256: 2882a81c586f41a94cc19187a3d522ba51829b63b6b33ab52772cb8c7515502b

```
"""Administrator-only accounts and audit history."""
from flask import Blueprint, g, request
from .auth import auth_required
from .common import APIError, audit, body, boolean, choice, db, oid, page_result, pagination, query_args, response

bp = Blueprint('admin', __name__)

@bp.get('/users')
@auth_required(admin=True)
def users():
    query_args({'page', 'page_size'})
    page, size = pagination()
    rows = list(db().users.find().sort('_id', 1).skip((page-1)*size).limit(size))
    return response(page_result(rows, db().users.count_documents({}), page, size))

@bp.get('/users/<user_id>')
@auth_required(admin=True)
def user_detail(user_id):
    user = db().users.find_one({'_id': oid(user_id)})
    if not user:
        raise APIError(404, 'User not found.')
    return response({'data': user})

@bp.route('/users/<user_id>', methods=['PUT', 'PATCH'])
@auth_required(admin=True)
def update_user(user_id):
    identifier = oid(user_id)
    if identifier == g.user['_id']:
        raise APIError(409, 'Administrators cannot change their own role or activation state.')
    data = body({'role', 'active'})
    if not data:
        raise APIError(400, 'Provide role or active.')
    if 'role' in data:
        choice(data['role'], 'role', ('admin', 'researcher'))
    if 'active' in data:
        boolean(data['active'], 'active')
    result = db().users.update_one({'_id': identifier}, {'$set': data, '$inc': {'auth_version': 1}})
    if not result.matched_count:
        raise APIError(404, 'User not found.')
    audit('permissions_updated', 'users', identifier)
    return response({'data': db().users.find_one({'_id': identifier})})

@bp.get('/audit-logs')
@auth_required(admin=True)
def audit_logs():
    query_args({'page', 'page_size'})
    page, size = pagination()
    rows = list(db().audit_logs.find().sort([('created_at', -1), ('_id', -1)]).skip((page-1)*size).limit(size))
    return response(page_result(rows, db().audit_logs.count_documents({}), page, size))
```

5. brainage/analytics.py

SHA-256: 5081638fecf2ddd50ba72ac57dfcff060743cdd691e81014f769c379d9b4a0be

```
"""MongoDB aggregation: cohorts and latest per-modality precomputed comparisons."""
import csv
import io
from flask import Blueprint, Response, request
from .auth import auth_required
from .common import choice, db, query_args, response, string
from .samples import FILTERS, filters

bp = Blueprint('analytics', __name__)

def age_group():
    return {'$switch': {'branches': [
        {'case': {'$eq': ['$age_at_scan_weeks', None]}, 'then': 'unknown'},
        {'case': {'$lt': ['$age_at_scan_weeks', 32]}, 'then': '<32 weeks'},
        {'case': {'$lt': ['$age_at_scan_weeks', 37]}, 'then': '32-<37 weeks'},
        {'case': {'$lt': ['$age_at_scan_weeks', 42]}, 'then': '37-<42 weeks'}], 'default': '>=42 weeks'}}

def cohort_rows(query):
    return list(db().samples.aggregate([{'$match': query}, {'$group': {
        '_id': {'age_group': age_group(), 'source': '$source'}, 'sample_count': {'$sum': 1},
        'dmri_count': {'$sum': {'$cond': ['$has_dmri', 1, 0]}},
        'fmri_count': {'$sum': {'$cond': ['$has_fmri', 1, 0]}},
        'mean_scan_age_weeks': {'$avg': '$age_at_scan_weeks'}}}, {'$sort': {'_id.source': 1, '_id.age_group': 1}}]))

def result_filter(names):
    return {"analyses." + field: string(request.args[param], param, 120)
        for param, field in names.items() if param in request.args}

def maturity_rows(query, selected=None, group_by="overall"):
    # Choose the latest result for each sample and modality, so repeated analyses do not overweight a sample.
    groups = {'modality': '$_id.modality', 'source': '$source'}
    if group_by == 'age_group':
        groups['age_group'] = '$age_group'
    return list(db().samples.aggregate([{'$match': query}, {'$set': {'age_group': age_group()}}, {'$unwind':
  '$analyses'},
        {'$match': selected or {}}, {'$sort': {'analyses.created_at': -1, 'analyses._id': -1}},
        {'$group': {'_id': {'sample': '$_id', 'modality': '$analyses.modality'},
            'result': {'$first': '$analyses'}, 'age_group': {'$first': '$age_group'}, 'source': {'$first':
  '$source'}}},
        {'$group': {'_id': groups,
            'sample_count': {'$sum': 1}, 'difference_count': {'$sum': {'$cond': [{'$ne':
  ['$result.age_difference_weeks', None]}, 1, 0]}}, 'mean_estimated_age_weeks': {'$avg': '$result.estimated_age_weeks'},
            'mean_age_difference_weeks': {'$avg': '$result.age_difference_weeks'}}},
        {'$sort': {'_id.source': 1, '_id.modality': 1}}]))

def comparison_rows(query, selected=None):
    return list(db().samples.aggregate([{'$match': query}, {'$unwind': '$analyses'},
        {'$match': {'analyses.modality': {'$in': ['dmri', 'fmri']}}},
        {'$match': selected or {}},
        {'$sort': {'analyses.created_at': -1, 'analyses._id': -1}},
        {'$group': {'_id': {'sample': '$_id', 'modality': '$analyses.modality'},
            'estimate': {'$first': '$analyses.estimated_age_weeks'}, 'source': {'$first': '$source'}}},
        {'$group': {'_id': '$_id.sample', 'source': {'$first': '$source'},
            'results': {'$push': {'k': '$_id.modality', 'v': '$estimate'}}}},
        {'$match': {'results': {'$size': 2}}}, {'$set': {'pair': {'$arrayToObject': '$results'}}},
        {'$group': {'_id': '$source', 'paired_sample_count': {'$sum': 1},
            'mean_dmri_minus_fmri_weeks': {'$avg': {'$subtract': ['$pair.dmri', '$pair.fmri']}}}},
        {'$sort': {'_id': 1}}]))

@bp.get('/statistics/overview')
@auth_required()
def overview():
    query_args(FILTERS)
    result = list(db().samples.aggregate([{'$match': filters()}, {'$facet': {
        'total': [{'$count': 'count'}],
        'by_status': [{'$group': {'_id': '$status', 'count': {'$sum': 1}}}, {'$sort': {'_id': 1}}],
        'by_source': [{'$group': {'_id': '$source', 'count': {'$sum': 1}}}, {'$sort': {'_id': 1}}],
        'modalities': [{'$group': {'_id': None, 'dmri': {'$sum': {'$cond': ['$has_dmri', 1, 0]}},
            'fmri': {'$sum': {'$cond': ['$has_fmri', 1, 0]}},
            'both': {'$sum': {'$cond': [{'$and': ['$has_dmri', '$has_fmri']}, 1, 0]}}}}]}}]))[0]
    result['total'] = result['total'][0]['count'] if result['total'] else 0
    return response({'data': result})

@bp.get('/statistics/cohorts')
@auth_required()
def cohorts():
```

```
    query_args(FILTERS)
    return response({'data': cohort_rows(filters()), 'unit': 'postmenstrual weeks at scan'})

@bp.get('/statistics/maturity')
@auth_required()
def maturity():
    query_args(FILTERS | {'method', 'model_version', 'group_by'})
    selected = result_filter({'method': 'method', 'model_version': 'model_version'})
    group_by = choice(request.args.get('group_by', 'overall'), 'group_by', ('overall', 'age_group'))
    return response({'data': maturity_rows(filters(), selected, group_by), 'selection': {k: v for k, v in request.args.items() if k in ('method', 'model_version', 'group_by')}, 'interpretation': 'Descriptive summary of supplied results; no diagnostic classification. Unfiltered models may be mixed.'})

@bp.get('/statistics/comparison')
@auth_required()
def comparison():
    keys = {'dmri_method', 'dmri_model_version', 'fmri_method', 'fmri_model_version'}
    query_args(FILTERS | keys)
    alternatives = []
    for mode in ('dmri', 'fmri'):
        selected = result_filter({mode + '_method': 'method', mode + '_model_version': 'model_version'})
        alternatives.append({'analyses.modality': mode, **selected})
    return response({'data': comparison_rows(filters(), {'$or': alternatives}), 'selection': {k: v for k, v in request.args.items() if k in keys}, 'comparison': 'Latest matching dMRI minus latest matching fMRI estimate for paired samples only; unfiltered models may be mixed.'})

@bp.get('/exports/cohorts')
@auth_required()
def export_cohorts():
    query_args(FILTERS | {'format'})
    fmt = choice(request.args.get('format', 'json'), 'format', ('json', 'csv'))
    rows = cohort_rows(filters())
    if fmt == 'json':
        return response({'data': rows})
    buffer = io.StringIO(newline='')
    writer = csv.writer(buffer)
    writer.writerow(['age_group', 'source', 'sample_count', 'dmri_count', 'fmri_count', 'mean_scan_age_weeks'])
    for row in rows:
        writer.writerow([row['_id']['age_group'], row['_id']['source'], row['sample_count'], row['dmri_count'], row['fmri_count'], row['mean_scan_age_weeks']])
    return Response(buffer.getvalue(), mimetype='text/csv', headers={'Content-Disposition': 'attachment; filename="cohorts.csv"'})
```

6. brainage/auth.py

SHA-256: 2264d2031eb7f5e5e314e563ea88e567345cc6f6ebb0c305b586c251826f46e5

```
"""JWT authentication, bcrypt password hashing, logout revocation and account checks."""
from datetime import timedelta, datetime, timezone
from functools import wraps
from uuid import uuid4
import re
import bcrypt
import jwt
from flask import Blueprint, current_app, g, request
from .common import APIError, audit, body, db, now, oid, response, string

bp = Blueprint('auth', __name__)

def password(value):
    # bcrypt accepts at most 72 bytes, not 72 Unicode characters.
    if not isinstance(value, str) or len(value) < 10 or len(value.encode('utf-8')) > 72:
        raise APIError(400, 'Password must have at least 10 characters and at most 72 UTF-8 bytes.')
    return value

def hash_password(value):
    return bcrypt.hashpw(password(value).encode(), bcrypt.gensalt()).decode()

def username(value):
    value = string(value, 'username', 40, 3).lower()
    if not re.fullmatch(r'[a-z0-9_.-]+', value):
        raise APIError(400, 'username may contain letters, numbers, _, . and - only.')
    return value

def auth_required(admin=False):
    def decorate(fn):
        @wraps(fn)
        def wrapped(*args, **kwargs):
            header = request.headers.get('Authorization', '')
            if not header.startswith('Bearer '):
                raise APIError(401, 'A Bearer token is required.')
            try:
                claims = jwt.decode(header[7:], current_app.config['JWT_SECRET'], algorithms=['HS256'],
                    issuer='brainage-connect', audience='brainage-api',
                    options={'require': ['exp', 'iat', 'sub', 'jti', 'ver']})
                user_id = oid(claims['sub'])
            except (jwt.PyJWTError, APIError, TypeError, KeyError):
                raise APIError(401, 'Invalid or expired token.')
            user = db().users.find_one({'_id': user_id, 'active': True})
            if not user or user['auth_version'] != claims['ver'] or db().revoked_tokens.find_one({'jti':
claims['jti']}):
                raise APIError(401, 'Session is no longer valid. Please log in again.')
            if admin and user['role'] != 'admin':
                raise APIError(403, 'Administrator access is required.')
            g.user, g.claims = user, claims
            return fn(*args, **kwargs)
        return wrapped
    return decorate

@bp.post('/auth/register')
def register():
    """New registrations are inactive until an administrator approves access."""
    data = body({'username', 'password', 'display_name'}, {'username', 'password', 'display_name'})
    user = {'username': username(data['username']), 'password_hash': hash_password(data['password']),
        'display_name': string(data['display_name'], 'display_name', 80), 'role': 'researcher',
        'active': False, 'auth_version': 0, 'created_at': now()}
    db().users.insert_one(user)
    return response({'data': user, 'message': 'Account created. Administrator approval is required.'}, 201)

@bp.post('/auth/login')
def login():
    data = body({'username', 'password'}, {'username', 'password'})
    name, secret = username(data['username']), password(data['password'])
    user = db().users.find_one({'username': name})
    if not user or not bcrypt.checkpw(secret.encode(), user['password_hash'].encode()):
        raise APIError(401, 'Invalid credentials.')
    if not user['active']:
        raise APIError(403, 'Account is inactive or awaiting administrator approval.')
    timestamp = now()
    token = jwt.encode({'sub': str(user['_id']), 'jti': str(uuid4()), 'iat': timestamp,
        'exp': timestamp + timedelta(seconds=current_app.config['TOKEN_SECONDS']),
        'iss': 'brainage-connect', 'aud': 'brainage-api', 'ver': user['auth_version']},
        current_app.config['JWT_SECRET'], algorithm='HS256')
```

```
    return response({'access_token': token, 'token_type': 'Bearer',
        'expires_in': current_app.config['TOKEN_SECONDS'], 'user': user})

@bp.post('/auth/logout')
@auth_required()
def logout():
    db().revoked_tokens.update_one({'jti': g.claims['jti']}, {'$set': {
        'expires_at': datetime.fromtimestamp(g.claims['exp'], timezone.utc)}}, upsert=True)
    return '', 204

@bp.get('/auth/me')
@auth_required()
def me():
    return response({'data': g.user})

@bp.put('/auth/password')
@auth_required()
def change_password():
    data = body({'current_password', 'new_password'}, {'current_password', 'new_password'})
    old = password(data['current_password'])
    if not bcrypt.checkpw(old.encode(), g.user['password_hash'].encode()):
        raise APIError(401, 'Current password is incorrect.')
    db().users.update_one({'_id': g.user['_id']}, {'$set': {'password_hash': hash_password(data['new_password'])},
        '$inc': {'auth_version': 1}})
    audit('password_changed', 'users', g.user['_id'])
    return response({'message': 'Password changed. All existing sessions are now invalid.'})
```

7. brainage/common.py

SHA-256: 9ebed41ef826495e2913d6cf23810ca0db97ba87d0802cb4b32fc8cefcc5edcf

```
"""Shared serialization, validation, access control and optimistic writes."""
import math
import re
from datetime import datetime, timezone
from bson import ObjectId
from flask import current_app, g, jsonify, request

class APIError(Exception):
    def __init__(self, status, message):
        self.status, self.message = status, message

def db():
    return current_app.extensions['mongo_db']

def now():
    return datetime.now(timezone.utc)

def clean(value):
    if isinstance(value, ObjectId):
        return str(value)
    if isinstance(value, datetime):
        return value.replace(tzinfo=timezone.utc).isoformat().replace('+00:00', 'Z')
    if isinstance(value, dict):
        return {k: clean(v) for k, v in value.items() if k not in ('password_hash', 'auth_version')}
    if isinstance(value, (list, tuple)):
        return [clean(v) for v in value]
    return value

def response(value, status=200):
    return jsonify(clean(value)), status

def oid(value):
    if not isinstance(value, str) or not re.fullmatch(r'[a-fA-F0-9]{24}', value):
        raise APIError(400, 'Invalid ObjectId: expected 24 hexadecimal characters.')
    return ObjectId(value)

def body(allowed, required=()):
    if not request.is_json:
        raise APIError(415, 'Content-Type must be application/json.')
    data = request.get_json()
    if not isinstance(data, dict):
        raise APIError(400, 'The request body must be a JSON object.')
    extra = set(data) - set(allowed)
    missing = set(required) - set(data)
    if extra:
        raise APIError(400, 'Unknown or read-only fields: ' + ', '.join(sorted(extra)))
    if missing:
        raise APIError(400, 'Missing fields: ' + ', '.join(sorted(missing)))
    return data

def string(value, name, maximum=200, minimum=1):
    if not isinstance(value, str) or not minimum <= len(value.strip()) <= maximum:
        raise APIError(400, f'{name} must be a string of {minimum}-{maximum} characters.')
    return value.strip()

def number(value, name, minimum, maximum, nullable=False):
    if value is None and nullable:
        return None
    if isinstance(value, bool) or not isinstance(value, (int, float)) or not math.isfinite(value) or not minimum <=
value <= maximum:
        raise APIError(400, f'{name} must be a finite number between {minimum} and {maximum}.')
    return value

def choice(value, name, choices):
    if not isinstance(value, str) or value not in choices:
        raise APIError(400, f'{name} must be one of: ' + ', '.join(choices))
    return value

def boolean(value, name):
    if not isinstance(value, bool):
        raise APIError(400, f'{name} must be a JSON boolean.')
    return value

def query_args(allowed):
    if set(request.args) - set(allowed):
        raise APIError(400, 'Unknown query parameter.')
```

```
    if any(len(request.args.getlist(k)) != 1 for k in request.args):
        raise APIError(400, 'Query parameters must not be repeated.')

def pagination():
    try:
        page = int(request.args.get('page', '1'))
        size = int(request.args.get('page_size', '20'))
    except ValueError:
        raise APIError(400, 'page and page_size must be integers.')
    if not 1 <= page <= 1000000 or not 1 <= size <= 100:
        raise APIError(400, 'page must be 1-1000000; page_size must be 1-100.')
    return page, size

def page_result(items, total, page, size):
    return {'data': items, 'pagination': {'page': page, 'page_size': size,
        'total': total, 'pages': math.ceil(total / size)}}

def visibility():
    if g.user['role'] == 'admin':
        return {}
    return {'$or': [{'owner_id': g.user['_id']}, {'visibility': 'shared'}]}

def sample(sample_id, write=False):
    item = db().samples.find_one({'$and': [{'_id': oid(sample_id)}, visibility()]})
    if not item:
        raise APIError(404, 'Sample not found.')
    if write and g.user['role'] != 'admin' and item['owner_id'] != g.user['_id']:
        raise APIError(403, 'Only the sample owner or an administrator may modify this record.')
    return item

def audit(action, resource, resource_id):
    db().audit_logs.insert_one({'actor_id': g.user['_id'], 'action': action,
        'resource': resource, 'resource_id': str(resource_id), 'created_at': now()})

def update_sample(original, changes, action):
    # Compare-and-swap prevents lost updates when nested records are edited concurrently.
    result = db().samples.update_one({'_id': original['_id'], 'version': original['version']},
        {'$set': {**changes, 'updated_at': now()}, '$inc': {'version': 1}})
    if not result.matched_count:
        raise APIError(409, 'The sample changed concurrently. Fetch it again and retry.')
    audit(action, 'samples', original['_id'])
    return db().samples.find_one({'_id': original['_id']})
```

8. brainage/samples.py

SHA-256: 5b76b6e24776d6179151056764849052 75c5c4dd7ef21309ee0282725f267d9a

```
"""Sample and embedded analysis/note resources; all reads enforce visibility."""
import re
from bson import ObjectId
from flask import Blueprint, g, request
from .auth import auth_required
from .common import (APIError, audit, body, boolean, choice, db, now, number, oid,
    page_result, pagination, query_args, response, sample, string, update_sample, visibility)

bp = Blueprint('samples', __name__)
SAMPLE_FIELDS = {'sample_code', 'age_at_scan_weeks', 'birth_age_weeks', 'sex',
    'has_dmri', 'has_fmri', 'status', 'visibility', 'description'}
FILTERS = {'q', 'age_min', 'age_max', 'modality', 'status', 'source', 'sex'}

def validate_sample(data, old=None):
    result = dict(data)
    for key in ('sample_code', 'description'):
        if key in result:
            result[key] = string(result[key], key, 120 if key == 'sample_code' else 2000, 1 if key == 'sample_code'
else 0)
    if 'sample_code' in result and not re.fullmatch(r'[A-Za-z0-9_.-]+', result['sample_code']):
        raise APIError(400, 'sample_code may contain letters, numbers, _, . and - only.')
    for key, low, high in [('age_at_scan_weeks', 20, 80), ('birth_age_weeks', 20, 45)]:
        if key in result:
            result[key] = number(result[key], key, low, high, nullable=True)
    for key in ('has_dmri', 'has_fmri'):
        if key in result:
            result[key] = boolean(result[key], key)
    for key, options in [('sex', ('Female', 'Male', 'Unknown')),
        ('status', ('registered', 'ready', 'processing', 'completed', 'failed')),
        ('visibility', ('private', 'shared'))]:
        if key in result:
            result[key] = choice(result[key], key, options)
    combined = {**(old or {}), **result}
    birth, scan = combined.get('birth_age_weeks'), combined.get('age_at_scan_weeks')
    if birth is not None and scan is not None and birth > scan:
        raise APIError(400, 'birth_age_weeks cannot exceed age_at_scan_weeks.')
    if old and old.get('source') == 'dhcp_local' and any(k in data for k in ('sample_code', 'has_dmri', 'has_fmri')):
        raise APIError(400, 'Imported sample identity and local file availability are managed by the importer.')
    if old:
        for analysis_entry in old.get('analyses', []):
            mode = analysis_entry['modality']
            if ((mode in ('dmri', 'combined') and not combined.get('has_dmri')) or
                (mode in ('fmri', 'combined') and not combined.get('has_fmri'))):
                raise APIError(409, 'Remove dependent analyses before disabling a modality.')
    return result

def filters():
    clauses = [visibility()]
    if 'q' in request.args:
        term = string(request.args['q'], 'q', 120)
        clauses.append({'sample_code': {'$regex': re.escape(term), '$options': 'i'}})
    limits = {}
    for name, operator in [('age_min', '$gte'), ('age_max', '$lte')]:
        if name in request.args:
            try:
                value = float(request.args[name])
            except ValueError:
                raise APIError(400, name + ' must be numeric.')
            limits[operator] = number(value, name, 20, 80)
    if limits:
        if limits.get('$gte', 20) > limits.get('$lte', 80):
            raise APIError(400, 'age_min must not exceed age_max.')
        clauses.append({'age_at_scan_weeks': limits})
    for name, options in [('status', ('registered', 'ready', 'processing', 'completed', 'failed')),
        ('source', ('dhcp_local', 'manual', 'synthetic')), ('sex', ('Female', 'Male', 'Unknown'))]:
        if name in request.args:
            clauses.append({name: choice(request.args[name], name, options)})
    if 'modality' in request.args:
        mode = choice(request.args['modality'], 'modality', ('dmri', 'fmri', 'both', 'neither'))
        clauses.append({'dmri': {'has_dmri': True}, 'fmri': {'has_fmri': True},
            'both': {'has_dmri': True, 'has_fmri': True}, 'neither': {'has_dmri': False, 'has_fmri': False}}[mode])
    return {'$and': clauses}

def sort_order(allowed, default):
    key = choice(request.args.get('sort', default), 'sort', allowed)
    order = choice(request.args.get('order', 'asc'), 'order', ('asc', 'desc'))
```

```
    return key, 1 if order == 'asc' else -1

@bp.get('/samples')
@auth_required()
def list_samples():
    query_args(FILTERS | {'page', 'page_size', 'sort', 'order'})
    page, size = pagination()
    key, direction = sort_order(('sample_code', 'age_at_scan_weeks', 'created_at', 'updated_at'), 'sample_code')
    query = filters()
    projection = {'notes': 0, 'analyses': 0, 'files': 0, 'source_metadata': 0}
    items = list(db().samples.find(query, projection).sort([(key, direction), ('_id', 1)]).skip((page - 1) *
size).limit(size))
    return response(page_result(items, db().samples.count_documents(query), page, size))

@bp.post('/samples')
@auth_required()
def create_sample():
    data = validate_sample(body(SAMPLE_FIELDS, {'sample_code', 'age_at_scan_weeks'}))
    item = {'_id': ObjectId(), 'owner_id': g.user['_id'], 'source': 'manual', 'visibility': 'private',
        'sex': 'Unknown', 'birth_age_weeks': None, 'has_dmri': False, 'has_fmri': False,
        'status': 'registered', 'description': '', 'notes': [], 'analyses': [], 'files': [],
        'created_at': now(), 'updated_at': now(), 'version': 1, **data}
    db().samples.insert_one(item)
    audit('created', 'samples', item['_id'])
    return response({'data': item}, 201)

@bp.get('/samples/<sample_id>')
@auth_required()
def get_sample(sample_id):
    return response({'data': sample(sample_id)})

@bp.route('/samples/<sample_id>', methods=['PUT', 'PATCH'])
@auth_required()
def edit_sample(sample_id):
    original = sample(sample_id, write=True)
    data = body(SAMPLE_FIELDS)
    if not data:
        raise APIError(400, 'At least one editable field is required.')
    changes = validate_sample(data, original)
    # Recompute differences if the reference age changes; never leave stale derived values.
    if 'age_at_scan_weeks' in changes:
        analyses = original['analyses']
        for result in analyses:
            result['age_difference_weeks'] = difference(result['estimated_age_weeks'], changes['age_at_scan_weeks'])
        changes['analyses'] = analyses
    return response({'data': update_sample(original, changes, 'updated')})

@bp.delete('/samples/<sample_id>')
@auth_required()
def delete_sample(sample_id):
    original = sample(sample_id, write=True)
    result = db().samples.delete_one({'_id': original['_id'], 'version': original['version']})
    if not result.deleted_count:
        raise APIError(409, 'The sample changed concurrently. Retry after fetching it.')
    audit('deleted', 'samples', original['_id'])
    return '', 204

def difference(estimate, age):
    return None if age is None else round(estimate - age, 6)

ANALYSIS_FIELDS = {'modality', 'estimated_age_weeks', 'method', 'model_version', 'provenance', 'quality_score'}
NOTE_FIELDS = {'text', 'category'}

def validate_child(kind, data, parent, original=None):
    merged = {**(original or {}), **data}
    if kind == 'notes':
        merged['text'] = string(merged.get('text'), 'text', 4000)
        merged['category'] = choice(merged.get('category', 'research'), 'category', ('research', 'quality', 'review'))
    else:
        merged['modality'] = choice(merged.get('modality'), 'modality', ('dmri', 'fmri', 'combined'))
        mode = merged['modality']
        if ((mode in ('dmri', 'combined') and not parent['has_dmri']) or
            (mode in ('fmri', 'combined') and not parent['has_fmri'])):
            raise APIError(400, 'The requested modality is not available for this sample.')
        merged['estimated_age_weeks'] = number(merged.get('estimated_age_weeks'), 'estimated_age_weeks', 20, 80)
        for key in ('method', 'model_version', 'provenance'):
            merged[key] = string(merged.get(key), key, 1000 if key == 'provenance' else 120)
        merged['quality_score'] = number(merged.get('quality_score'), 'quality_score', 0, 1, nullable=True)
        merged['age_difference_weeks'] = difference(merged['estimated_age_weeks'], parent['age_at_scan_weeks'])
        merged['result_type'] = 'synthetic_demonstration' if parent['source'] == 'synthetic' else
'user_supplied_precomputed'
```

```
    return merged

def child_owner(item):
    if g.user['role'] != 'admin' and item['author_id'] != g.user['_id']:
        raise APIError(403, 'Only the author or an administrator may modify this entry.')

# The same resource workflow is used for both independently named embedded collections.
def register_children(kind):
    path = '/samples/<sample_id>/' + kind
    allowed = NOTE_FIELDS if kind == 'notes' else ANALYSIS_FIELDS
    required = {'text'} if kind == 'notes' else ANALYSIS_FIELDS - {'quality_score'}

    @auth_required()
    def listing(sample_id):
        query_args({'page', 'page_size', 'sort', 'order'})
        parent = sample(sample_id)
        page, size = pagination()
        key, direction = sort_order(('created_at', 'updated_at'), 'created_at')
        entries = sorted(parent[kind], key=lambda x: (x[key], str(x['_id'])), reverse=direction == -1)
        return response(page_result(entries[(page-1)*size:page*size], len(entries), page, size))

    @auth_required()
    def create(sample_id):
        parent = sample(sample_id)
        data = body(allowed, required)
        if len(parent[kind]) >= 200:
            raise APIError(409, 'This sample already has the maximum of 200 entries in this collection.')
        entry = validate_child(kind, data, parent)
        entry.update(_id=ObjectId(), author_id=g.user['_id'], created_at=now(), updated_at=now())
        update_sample(parent, {kind: parent[kind] + [entry]}, kind + '_created')
        return response({'data': entry}, 201)

    @auth_required()
    def detail(sample_id, child_id):
        parent = sample(sample_id)
        identifier = oid(child_id)
        entry = next((x for x in parent[kind] if x['_id'] == identifier), None)
        if not entry:
            raise APIError(404, 'Entry not found.')
        if request.method == 'GET':
            return response({'data': entry})
        child_owner(entry)
        if request.method == 'DELETE':
            entries = [x for x in parent[kind] if x['_id'] != identifier]
            update_sample(parent, {kind: entries}, kind + '_deleted')
            return '', 204
        data = body(allowed)
        if not data:
            raise APIError(400, 'At least one editable field is required.')
        updated = validate_child(kind, data, parent, entry)
        updated['updated_at'] = now()
        entries = [updated if x['_id'] == identifier else x for x in parent[kind]]
        update_sample(parent, {kind: entries}, kind + '_updated')
        return response({'data': updated})

    bp.add_url_rule(path, kind + '_list', listing, methods=['GET'])
    bp.add_url_rule(path, kind + '_create', create, methods=['POST'])
    bp.add_url_rule(path + '/<child_id>', kind + '_detail', detail, methods=['GET', 'PUT', 'PATCH', 'DELETE'])

register_children('notes')
register_children('analyses')
```

9. scripts/__init__.py

SHA-256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

10. scripts/bootstrap.py

SHA-256: 2fd00c56f7e89646c55dd7ed88c6397fd92d17878f9c0c24d9de68d6cdae5e15

```
"""Create local configuration and a single administrator without overwriting existing secrets."""
import secrets
from brainage import ROOT, create_app
from brainage.auth import hash_password
from brainage.common import now

def main():
    path = ROOT / '.env'
    if not path.exists():
        path.write_text('MONGO_URI=mongodb://127.0.0.1:27018\nMONGO_DB=brainage_connect\nJWT_SECRET=' +
            secrets.token_urlsafe(48) + '\nPORT=5000\n', encoding='utf-8')
    app = create_app()
    db = app.extensions['mongo_db']
    if not db.users.find_one({'role': 'admin', 'active': True}):
        secret = secrets.token_urlsafe(15)
        db.users.insert_one({'username': 'admin', 'password_hash': hash_password(secret),
            'display_name': 'Project Administrator', 'role': 'admin', 'active': True,
            'auth_version': 0, 'created_at': now()})
        (ROOT / 'docs' / 'LOCAL_ACCESS.txt').write_text(
            'Local development only. Do not submit or share this file.\nURL: http://127.0.0.1:5000\n'
            'Username: admin\nPassword: ' + secret + '\nChange using PUT /api/v1/auth/password.\n', encoding='utf-8')
    print('Configuration and administrator ready. Credentials: docs/LOCAL_ACCESS.txt')

if __name__ == '__main__':
    main()
```

11. scripts/export_database.py

SHA-256: d997b034dd47a66857d99d62dcd934ea24cd92ae45a433820bc05a1dc4bd1487

```
"""Export actual MongoDB collections with mongoexport (Canonical Extended JSON)."""
import argparse
import shutil
import subprocess
from pathlib import Path
from brainage import ROOT, create_app

COLLECTIONS = ('samples', 'users', 'audit_logs', 'revoked_tokens')

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--database', default='brainage_connect')
    parser.add_argument('--output', default='exports/local')
    args = parser.parse_args()
    app = create_app({'MONGO_DB': args.database})
    executable = shutil.which('mongoexport') or next((str(x) for x in (ROOT / 'tools').rglob('mongoexport.exe')), None)
    if not executable:
        raise SystemExit('Install MongoDB Database Tools or place them under tools/.')
    target = (ROOT / args.output).resolve()
    target.mkdir(parents=True, exist_ok=True)
    for collection in COLLECTIONS:
        subprocess.run([executable, '--uri', app.config['MONGO_URI'], '--db', args.database,
            '--collection', collection, '--jsonArray', '--jsonFormat=canonical', '--pretty',
            '--out', str(target / (collection + '.json'))], check=True)
    print('Exported actual database collections to', target)

if __name__ == '__main__':
    main()
```

12. scripts/generate_openapi.py

SHA-256: f70967042a0683fc70f7f2096274abc603f036c3de704e5fab17301de5b80cd3

```
"""Generate a portable OpenAPI description matching the Flask route table."""
import json
import re
from brainage import ROOT, create_app

def shape(value):
    """Conservative schema from a redacted example; optional and nullable fields may vary."""
    if isinstance(value, dict):
        return {'type': 'object', 'properties': {k: shape(v) for k, v in value.items()}}
    if isinstance(value, list):
        return {'type': 'array', 'items': shape(value[0]) if value else {}}
    if value is None:
        return {'description': 'Observed null; see API.md for allowed values.'}
    kind = 'boolean' if isinstance(value, bool) else 'number' if isinstance(value, (int, float)) else 'string'
    return {'type': kind, 'nullable': True}

def response_examples():
    collection = json.loads((ROOT / 'postman/BrainAgeConnect.postman_collection.json').read_text(encoding='utf-8'))
    result = {}
    for item in collection['item']:
        for example in item.get('response', []):
            if example['code'] not in (200, 201) or 'application/json' not in str(example.get('header', [])):
                continue
            url = item['request']['url'].split('?', 1)[0].replace('{{base_url}}', '/api/v1')
            url = re.sub(r'\{\{[^}]+\}\}', '{}', url)
            try:
                body = json.loads(example['body'])
            except (ValueError, KeyError):
                continue
            result.setdefault((item['request']['method'], url, str(example['code'])), body)
    return result

def main():
    app = create_app()
    string = {'type': 'string'}
    age = {'type': 'number', 'minimum': 20, 'maximum': 80, 'nullable': True}
    sample = {'type': 'object', 'additionalProperties': False, 'properties': {
        'sample_code': {'type': 'string', 'pattern': '^[A-Za-z0-9_.-]+$', 'maxLength': 120},
        'age_at_scan_weeks': age, 'birth_age_weeks': {'type': 'number', 'minimum': 20, 'maximum': 45, 'nullable':
True},
        'sex': {'type': 'string', 'enum': ['Female', 'Male', 'Unknown']},
        'has_dmri': {'type': 'boolean'}, 'has_fmri': {'type': 'boolean'},
        'status': {'type': 'string', 'enum': ['registered', 'ready', 'processing', 'completed', 'failed']},
        'visibility': {'type': 'string', 'enum': ['private', 'shared']}, 'description': {'type': 'string', 'maxLength':
2000}}}
    note = {'type': 'object', 'additionalProperties': False, 'properties': {
        'text': {'type': 'string', 'minLength': 1, 'maxLength': 4000},
        'category': {'type': 'string', 'enum': ['research', 'quality', 'review']}}}
    analysis = {'type': 'object', 'additionalProperties': False, 'properties': {
        'modality': {'type': 'string', 'enum': ['dmri', 'fmri', 'combined']},
        'estimated_age_weeks': {'type': 'number', 'minimum': 20, 'maximum': 80},
        'method': {'type': 'string', 'minLength': 1, 'maxLength': 120},
        'model_version': {'type': 'string', 'minLength': 1, 'maxLength': 120},
        'provenance': {'type': 'string', 'minLength': 1, 'maxLength': 1000},
        'quality_score': {'type': 'number', 'minimum': 0, 'maximum': 1, 'nullable': True}}}
    schemas = {'SampleInput': sample, 'NoteInput': note, 'AnalysisInput': analysis,
        'Login': {'type': 'object', 'additionalProperties': False, 'required': ['username', 'password'],
            'properties': {'username': string, 'password': {'type': 'string', 'format': 'password'}}},
        'Register': {'type': 'object', 'additionalProperties': False, 'required': ['username', 'password',
'display_name'],
            'properties': {'username': string, 'password': {'type': 'string', 'format': 'password'}, 'display_name':
string}},
        'PasswordChange': {'type': 'object', 'additionalProperties': False, 'required': ['current_password',
'new_password'],
            'properties': {'current_password': {'type': 'string', 'format': 'password'}, 'new_password': {'type':
'string', 'format': 'password'}}},
        'UserUpdate': {'type': 'object', 'additionalProperties': False, 'minProperties': 1,
            'properties': {'active': {'type': 'boolean'}, 'role': {'type': 'string', 'enum': ['admin',
'researcher']}}},
        'Error': {'type': 'object', 'properties': {'error': {'type': 'object', 'properties': {
            'status': {'type': 'integer'}, 'message': string}}}}}
    document = {'openapi': '3.0.3', 'info': {'title': 'BrainAge Connect', 'version': '1.0.0',
        'description': 'Research metadata and precomputed-result management. No MRI-to-brain-age inference. All ages
are postmenstrual weeks. See API.md for permissions and examples.'},
        'servers': [{'url': 'http://127.0.0.1:5000'}, {'url': 'http://127.0.0.1:5001'}],
        'security': [{'BearerAuth': []}], 'paths': {}, 'components': {
```

```
            'securitySchemes': {'BearerAuth': {'type': 'http', 'scheme': 'bearer', 'bearerFormat': 'JWT'}}, 'schemas': schemas}}
    filters = {'q': string, 'age_min': {'type':'number','minimum':20,'maximum':80}, 'age_max': {'type':'number','minimum':20,'maximum':80},
        'modality': {'type':'string','enum':['dmri','fmri','both','neither']},
        'source': {'type':'string','enum':['dhcp_local','manual','synthetic']},
        'status': sample['properties']['status'], 'sex': sample['properties']['sex']}
    examples = response_examples()
    for rule in app.url_map.iter_rules():
        if not rule.rule.startswith('/api/v1/'):
            continue
        path = re.sub(r'<([^>]+)>', r'{\1}', rule.rule)
        for method in sorted(rule.methods - {'HEAD', 'OPTIONS'}):
            status = '204' if method == 'DELETE' or path.endswith('/logout') else '201' if method == 'POST' and not path.endswith('/login') else '200'
            operation = {'operationId': rule.endpoint.replace('.', '_') + '_' + method.lower(),
                'summary': rule.endpoint.split('.')[-1].replace('_', ' ').capitalize(),
                'description': 'Authenticated resource. PUT and PATCH update only supplied editable fields; omitted fields remain unchanged.' if method in ('PUT','PATCH') else 'See docs/API.md for data semantics and authorization.',
                'responses': {status: {'description': 'Success'}, **{str(code): {'description': text,
                    'content': {'application/json': {'schema': {'$ref': '#/components/schemas/Error'}}}}
                    for code,text in [(400,'Invalid input'),(401,'Missing/expired/revoked session'),(403,'Insufficient permission'),(404,'Not found or not visible'),(409,'Duplicate/concurrent conflict'),(415,'JSON content type required'),(503,'Database unavailable')]}}}
            if status != '204':
                example_key = (method, re.sub(r'\{[^}]+\}', '{}', path), status)
                example = examples.get(example_key)
                if example is None and method == 'PATCH':
                    example = examples.get(('PUT', example_key[1], status))
                schema = shape(example) if example is not None else {'type': 'object', 'description': 'JSON response; see API.md for fields.'}
                operation['responses'][status]['content'] = {'application/json': {'schema': schema}}
                if example is not None:
                    operation['responses'][status]['content']['application/json']['example'] = example
                    operation['responses'][status]['description'] += '; schema describes fields observed in a redacted synthetic example and is not exhaustive.'
            if path.endswith(('/login','/register','/health','/openapi.json')):
                operation['security'] = []
            parameters = [{'name': name, 'in':'path','required':True,'schema':{'type':'string','pattern':'^[a-fA-F0-9]{24}$'}} for name in rule.arguments]
            query = {}
            if method == 'GET':
                if path.endswith('/samples') or '/statistics/' in path or '/exports/' in path:
                    query.update(filters)
                if path.endswith(('/samples','/notes','/analyses','/users','/audit-logs')):
                    query.update(page={'type':'integer','minimum':1,'maximum':1000000,'default':1}, page_size={'type':'integer','minimum':1,'maximum':100,'default':20})
                if path.endswith('/samples'):
                    query['sort']={'type':'string','enum':['sample_code','age_at_scan_weeks','created_at','updated_at'],'default':'sample_code'}
                elif path.endswith(('/notes','/analyses')):
                    query['sort']={'type':'string','enum':['created_at','updated_at'],'default':'created_at'}
                if 'sort' in query:
                    query['order']={'type':'string','enum':['asc','desc'],'default':'asc'}
                if path.endswith('/statistics/maturity'):
                    query.update(method={'type':'string','maxLength':120}, model_version={'type':'string','maxLength':120}, group_by={'type':'string','enum':['overall','age_group'],'default':'overall'})
                if path.endswith('/statistics/comparison'):
                    query.update({name: {'type':'string','maxLength':120} for name in ('dmri_method','dmri_model_version','fmri_method','fmri_model_version')})
                if '/exports/' in path:
                    query['format']={'type':'string','enum':['json','csv'],'default':'json'}
                    operation['responses']['200']['content']={'application/json':{'schema':{'type':'object'}},'text/csv':{'schema':{'type':'string'}}}
            parameters.extend({'name':k,'in':'query','schema':v} for k,v in query.items())
            if parameters:
                operation['parameters']=parameters
            schema_name = None
            required=[]
            if method in ('POST','PUT','PATCH'):
                if '/notes' in path:
                    schema_name='NoteInput'; required=['text']
                elif '/analyses' in path:
                    schema_name='AnalysisInput'; required=['modality','estimated_age_weeks','method','model_version','provenance']
                elif '/samples' in path:
                    schema_name='SampleInput'; required=['sample_code','age_at_scan_weeks']
                elif path.endswith('/register'): schema_name='Register'
                elif path.endswith('/login'): schema_name='Login'
                elif path.endswith('/password'): schema_name='PasswordChange'
```

```
                elif '/users/' in path: schema_name='UserUpdate'
            if schema_name:
                schema={'allOf':[{'$ref':'#/components/schemas/'+schema_name}]}
                if method == 'POST' and required: schema['allOf'].append({'required':required})
                if method in ('PUT','PATCH'): schema['allOf'].append({'minProperties':1})
                operation['requestBody']={'required':True,'content':{'application/json':{'schema':schema}}}
            document['paths'].setdefault(path,{})[method.lower()]=operation
    (ROOT/'docs/openapi.json').write_text(json.dumps(document,indent=2),encoding='utf-8')
    print('OpenAPI operations:',sum(len(p) for p in document['paths'].values()))

if __name__ == '__main__':
    main()
```

13. scripts/import_database.py

SHA-256: d577fadd5f0d8f4f837f5e571153a6164cf936df377468bd7e444c7e3c3a9af0

```python
"""Restore exported collections into an empty database; never delete existing data."""
import argparse
import json
import shutil
import subprocess
from pathlib import Path
from brainage import ROOT, create_app, ensure_indexes
from scripts.export_database import COLLECTIONS

def restore_export(client, uri, database_name, directory):
    database = client[database_name]
    if any(database[name].count_documents({}) for name in database.list_collection_names()):
        raise RuntimeError('Target database is not empty; choose a new name. No data have been deleted.')
    executable = shutil.which('mongoimport') or next((str(p) for p in (ROOT / 'tools').rglob('mongoimport.exe')), None)
    if not executable:
        raise RuntimeError('MongoDB Database Tools (mongoimport) are required.')
    prepared = []
    for name in COLLECTIONS:
        path = Path(directory) / (name + '.json')
        records = json.loads(path.read_text(encoding='utf-8'))
        if not isinstance(records, list):
            raise ValueError(f'{path.name} must contain a JSON array.')
        prepared.append((name, path, len(records)))
    counts = {}
    for name, path, length in prepared:
        if length:
            subprocess.run([executable, '--uri', uri, '--db', database_name,
                '--collection', name, '--jsonArray', '--file', str(path)], check=True)
        elif name not in database.list_collection_names():
            # mongoimport rejects an empty JSON array; create its empty collection explicitly.
            database.create_collection(name)
        counts[name] = database[name].count_documents({})
        if counts[name] != length:
            raise RuntimeError(f'Import count mismatch for {name}.')
    ensure_indexes(database)
    return counts

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--database', default='brainage_restored')
    parser.add_argument('--input', type=Path, default=ROOT / 'exports/demo')
    args = parser.parse_args()
    app = create_app()
    print(restore_export(app.extensions['mongo_client'], app.config['MONGO_URI'], args.database, args.input))
    print('Set MONGO_DB in .env to the restored name to serve this database.')

if __name__ == '__main__':
    main()
```

14. scripts/import_dhcp.py

SHA-256: dbbdb3fa522012ed3e8dda877cb334bbb5cf1c6a00c11ff1ffe225b8b32bff9b

```
"""Read local BIDS metadata without moving MRI files or fabricating results.

Idempotent inserts: existing samples are never replaced. Use --refresh-files to refresh
only detected file inventories/availability. User edits, notes and analyses survive reruns.
"""
import argparse
import csv
import json
import math
from pathlib import Path
from bson import ObjectId
from brainage import ROOT, create_app
from brainage.common import now

def rows(path):
    if not path.exists():
        return []
    with path.open(encoding='utf-8-sig', newline='') as handle:
        return list(csv.DictReader(handle, delimiter='\t'))

def numeric(value):
    try:
        result = float(value)
        return result if math.isfinite(result) else None
    except (TypeError, ValueError):
        return None

def inventory(root, subject, session, modality):
    base = root / ('sub-' + subject) / ('ses-' + session)
    files = []
    available = False
    for path in sorted(base.rglob('*')) if base.exists() else []:
        if not path.is_file():
            continue
        size = path.stat().st_size
        primary = path.name.endswith('_desc-preproc_dwi.nii.gz') if modality == 'dmri' else
path.name.endswith('_task-rest_desc-preproc_bold.nii.gz')
        # Detect obvious LFS stubs and empty files. This is NOT full MRI quality control.
        signature_ok = False
        if primary and size > 1024:
            with path.open('rb') as handle:
                signature_ok = handle.read(2) == b'\x1f\x8b'
        available = available or signature_ok
        files.append({'_id': ObjectId(), 'modality': modality, 'relative_path': path.relative_to(root).as_posix(),
            'size_bytes': size, 'primary_image': primary, 'gzip_signature_valid': signature_ok if primary else None})
    return files, available

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--dmri', type=Path, default=Path.home() / 'Desktop/dhcp_dmri_pipeline')
    parser.add_argument('--fmri', type=Path, default=Path.home() / 'Desktop/dhcp_fmri_pipeline')
    parser.add_argument('--refresh-files', action='store_true')
    args = parser.parse_args()
    app = create_app()
    db = app.extensions['mongo_db']
    admin = db.users.find_one({'role': 'admin', 'active': True})
    if not admin:
        raise SystemExit('Run python -m scripts.bootstrap first.')
    merged, descriptions, conflicts = {}, {}, []
    for modality, root in [('dmri', args.dmri), ('fmri', args.fmri)]:
        if not (root / 'participants.tsv').is_file():
            raise SystemExit(f'Missing participants.tsv: {root}')
        descriptions[modality] = json.loads((root / 'dataset_description.json').read_text(encoding='utf-8'))
        participants = {r['participant_id'].removeprefix('sub-'): r for r in rows(root / 'participants.tsv')}
        for subject_dir in sorted(root.glob('sub-*')):
            if not subject_dir.is_dir():
                continue
            subject = subject_dir.name.removeprefix('sub-')
            participant = participants.get(subject, {})
            for session_row in rows(subject_dir / (subject_dir.name + '_sessions.tsv')):
                session = session_row['session_id'].removeprefix('ses-')
                code = f'sub-{subject}_ses-{session}'
                age = numeric(session_row.get('scan_age'))
                birth = numeric(participant.get('birth_age'))
                if age is not None and not 20 <= age <= 80:
                    conflicts.append({'sample_code': code, 'field': 'scan_age_out_of_range', 'value': age}); age = None
```

```
                if birth is not None and (not 20 <= birth <= 45 or (age is not None and birth > age)):
                    conflicts.append({'sample_code': code, 'field': 'birth_age_invalid', 'value': birth}); birth = None
                if code not in merged:
                    merged[code] = {'sample_code': code, 'subject_code': subject, 'session_code': session,
                        'age_at_scan_weeks': age, 'birth_age_weeks': birth,
                        'sex': participant.get('gender') if participant.get('gender') in ('Male', 'Female') else
    'Unknown',
                        'has_dmri': False, 'has_fmri': False, 'metadata_modalities': [], 'files': [],
    'source_metadata': {}}
                item = merged[code]
                if age is not None and item['age_at_scan_weeks'] is not None and abs(age - item['age_at_scan_weeks']) >
     0.00001:
                    conflicts.append({'sample_code': code, 'field': 'scan_age_conflict', 'values':
    [item['age_at_scan_weeks'], age]})
                    item['age_at_scan_weeks'] = None
                files, available = inventory(root, subject, session, modality)
                item['files'].extend(files)
                item['has_' + modality] = available
                item['metadata_modalities'].append(modality)
                item['source_metadata'][modality] = {'participant': participant, 'session': session_row}
    inserted = refreshed = skipped = 0
    for item in merged.values():
        existing = db.samples.find_one({'sample_code': item['sample_code']})
        if existing:
            if args.refresh_files:
                result = db.samples.update_one({'_id': existing['_id'], 'source': 'dhcp_local', 'version':
    existing['version']},
                    {'$set': {**{k: item[k] for k in ('files', 'has_dmri', 'has_fmri', 'metadata_modalities')},
    'updated_at': now()}, '$inc': {'version': 1}})
                refreshed += result.modified_count
            else:
                skipped += 1
            continue
        note = {'_id': ObjectId(), 'author_id': admin['_id'], 'category': 'quality',
            'text': 'Importer observation: metadata available for ' + ', '.join(item['metadata_modalities']) +
            '. File presence is not a quality assessment. No developmental age has been inferred.',
            'created_at': now(), 'updated_at': now()}
        item.update(_id=ObjectId(), owner_id=admin['_id'], source='dhcp_local', visibility='private',
            status='ready' if item['has_dmri'] or item['has_fmri'] else 'registered',
            description='Local dHCP research metadata; postmenstrual age in weeks.',
            notes=[note], analyses=[], created_at=now(), updated_at=now(), version=1)
        db.samples.insert_one(item)
        inserted += 1
    report = {'imported_at': now().isoformat(), 'discovered_sessions': len(merged),
        'inserted': inserted, 'refreshed': refreshed, 'skipped': skipped,
        'dmri_images_present': sum(x['has_dmri'] for x in merged.values()),
        'fmri_images_present': sum(x['has_fmri'] for x in merged.values()),
        'both_images_present': sum(x['has_dmri'] and x['has_fmri'] for x in merged.values()),
        'metadata_in_both': sum(len(x['metadata_modalities']) == 2 for x in merged.values()),
        'files_indexed': sum(len(x['files']) for x in merged.values()),
        'conflicts': conflicts, 'dataset_descriptions': descriptions,
        'note': 'Presence check only: size and gzip magic; no decompression, scan QC or brain-age inference
    performed.'}
    private = ROOT / 'data/private'
    private.mkdir(parents=True, exist_ok=True)
    (private / 'import_report.json').write_text(json.dumps(report, indent=2), encoding='utf-8')
    (private / 'source_paths.json').write_text(json.dumps({'dmri': str(args.dmri.resolve()), 'fmri':
    str(args.fmri.resolve())}, indent=2), encoding='utf-8')
    print(json.dumps({k: v for k, v in report.items() if k not in ('conflicts', 'dataset_descriptions')}, indent=2))

if __name__ == '__main__':
    main()
```

15. scripts/import_results.py

SHA-256: 01499810f03b661ec855100c269cebd7b4939e009f657267e4785d3387e61f70

```
"""Import precomputed results via the same authenticated, validated API used by clients.

Input JSON: [{"sample_id": "24-hex-id", "result": { ... analysis fields ... }}].
Token is read from BRAINAGE_TOKEN; it is never written to disk or logs.
No MRI inference or model training occurs here. Duplicate imports create new history entries.
"""
import argparse
import json
import os
import requests

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('file')
    parser.add_argument('--base-url', default='http://127.0.0.1:5000/api/v1')
    args = parser.parse_args()
    token = os.environ.get('BRAINAGE_TOKEN')
    if not token:
        raise SystemExit('Set BRAINAGE_TOKEN to a current authenticated session token.')
    with open(args.file, encoding='utf-8') as handle:
        items = json.load(handle)
    if not isinstance(items, list):
        raise SystemExit('Input must be a JSON array.')
    success = 0
    for index, item in enumerate(items):
        response = requests.post(args.base_url + '/samples/' + item['sample_id'] + '/analyses',
            json=item['result'], headers={'Authorization': 'Bearer ' + token}, timeout=30)
        if response.status_code != 201:
            raise SystemExit(f'Row {index+1} failed ({response.status_code}): {response.text}. {success} earlier rows were imported.')
        success += 1
    print(f'Imported {success} precomputed results.')

if __name__ == '__main__':
    main()
```

16. scripts/package_submission.py

SHA-256: 70b78c2b2aa069d43d37bf767794a1cac2d10354b25ab4351507e2179ce9fb8c

```
"""Build code and synthetic-data ZIPs. Never bundle local dHCP data or secrets."""
import subprocess
import sys
from zipfile import ZipFile, ZIP_DEFLATED
from brainage import ROOT, create_app

def main():
    target = ROOT / 'submission'
    target.mkdir(exist_ok=True)
    app = create_app({'MONGO_DB': 'brainage_demo'})
    database = app.extensions['mongo_db']
    if not database.samples.count_documents({}) or database.samples.count_documents({'source': {'$ne': 'synthetic'}}):
        raise SystemExit('Submission dataset must be a nonempty, exclusively synthetic database.')
    subprocess.run([sys.executable, '-m', 'scripts.export_database', '--database', 'brainage_demo',
        '--output', 'exports/demo'], check=True, cwd=ROOT)
    include_dirs = ['brainage', 'scripts', 'tests', 'postman', 'static']
    include_files = ['app.py', 'requirements.txt', 'requirements-dev.txt', '.env.example', '.gitignore',
        'README.md', '开始使用.md', 'START_PROJECT.cmd', 'START_DEMO.cmd', 'STOP_PROJECT.cmd', 'RUN_TESTS.cmd',
'setup.ps1', 'start.ps1',
        'stop.ps1', 'docs/API.md', 'docs/DESIGN_AND_AI.md', 'docs/DATA.md', 'docs/openapi.json',
        'docs/TEST_REPORT.md', 'docs/REQUIREMENTS_MAPPING.md', 'docs/SUBMISSION_DESIGN.md',
        'docs/SUBMISSION_STATUS.md', '项目交接文档.md']
    with ZipFile(target / 'BrainAgeConnect_Code.zip', 'w', ZIP_DEFLATED) as archive:
        for directory in include_dirs:
            for path in (ROOT / directory).rglob('*'):
                if path.is_file() and '__pycache__' not in path.parts and path.suffix != '.pyc':
                    archive.write(path, path.relative_to(ROOT))
        for name in include_files:
            path = ROOT / name
            if path.is_file():
                archive.write(path, name)
    with ZipFile(target / 'BrainAgeConnect_SyntheticDatabase.zip', 'w', ZIP_DEFLATED) as archive:
        for path in (ROOT / 'exports/demo').glob('*.json'):
            archive.write(path, path.name)
    print('Created code ZIP and synthetic database ZIP in submission/.')

if __name__ == '__main__':
    main()
```

17. scripts/run_postman.py

SHA-256: fe179cc150c1e40d35d55bd44d393cb33fbda68325e9c90814c4984d8212ff18

```
"""Run the exported Postman collection with Newman against an isolated real MongoDB."""
import os
import shutil
import subprocess
import sys
import time
import uuid
from pathlib import Path
import requests
from brainage import ROOT, create_app
from scripts.seed_demo import seed

def main():
    name = 'brainage_postman_' + uuid.uuid4().hex
    app = create_app({'MONGO_DB': name})
    seed(app.extensions['mongo_db'], 10)
    node = shutil.which('node') or str(Path.home() / '.cache/codex-runtimes/codex-primary-runtime/dependencies/node/bin/node.exe')
    newman = ROOT / 'tools/postman-runner/node_modules/newman/bin/newman.js'
    if not newman.is_file():
        raise SystemExit('Install newman with: npm install --prefix tools/postman-runner newman')
    env = {**os.environ, 'MONGO_DB': name, 'PORT': '5051'}
    runtime = ROOT / 'runtime'
    runtime.mkdir(exist_ok=True)
    log = (runtime / 'postman-api.log').open('w', encoding='utf-8')
    process = subprocess.Popen([sys.executable, 'app.py'], cwd=ROOT, env=env, stdout=log, stderr=log)
    try:
        for _ in range(40):
            if process.poll() is not None:
                raise RuntimeError('Isolated API failed to start; see runtime/postman-api.log.')
            try:
                if requests.get('http://127.0.0.1:5051/api/v1/health', timeout=1).status_code == 200:
                    break
            except requests.RequestException:
                pass
            time.sleep(.25)
        else:
            raise RuntimeError('Isolated API did not become ready.')
        result = subprocess.run([node, str(newman), 'run', str(ROOT / 'postman/BrainAgeConnect.postman_collection.json'),
            '--env-var', 'base_url=http://127.0.0.1:5051/api/v1', '--reporters', 'cli,json',
            '--reporter-json-export', str(runtime / 'newman-results.json')], cwd=ROOT)
        if result.returncode:
            raise SystemExit(result.returncode)
    finally:
        process.terminate()
        process.wait(timeout=15)
        log.close()
        assert name.startswith('brainage_postman_')
        app.extensions['mongo_client'].drop_database(name)
        app.extensions['mongo_client'].close()

if __name__ == '__main__':
    main()
```

18. scripts/seed_demo.py

SHA-256: 2af08cbe038c54ab1e6d1e7e67304e81c05e00324147a0cd077e4fac9ec16627

```
"""Seed a SEPARATE fully synthetic, reproducible database for sharing/testing."""
import argparse
import random
from bson import ObjectId
from brainage import create_app
from brainage.auth import hash_password
from brainage.common import now

def seed(db, count=150):
    if db.samples.count_documents({}) or db.users.count_documents({}):
        raise RuntimeError('Demo database is not empty; refusing to overwrite it.')
    admin = {'_id': ObjectId(), 'username': 'demo.admin', 'password_hash': hash_password('DemoAdmin!2026'),
        'display_name': 'Synthetic Demo Administrator', 'role': 'admin', 'active': True, 'auth_version': 0,
 'created_at': now()}
    researcher = {'_id': ObjectId(), 'username': 'demo.researcher', 'password_hash':
 hash_password('DemoResearch!2026'),
        'display_name': 'Synthetic Demo Researcher', 'role': 'researcher', 'active': True, 'auth_version': 0,
 'created_at': now()}
    db.users.insert_many([admin, researcher])
    rng = random.Random(661)
    items = []
    for i in range(count):
        age = round(rng.uniform(28, 45), 4)
        dmri, fmri = i % 5 != 0, i % 4 != 0
        analyses = []
        for mode, present in [('dmri', dmri), ('fmri', fmri)]:
            if present:
                estimate = round(age + rng.uniform(-2, 2), 4)
                analyses.append({'_id': ObjectId(), 'author_id': researcher['_id'], 'modality': mode,
                    'estimated_age_weeks': estimate, 'age_difference_weeks': round(estimate-age, 6),
                    'method': 'Synthetic fixture: reference age plus random offset', 'model_version': 'fixture-v1',
                    'provenance': 'Entirely invented for software testing. Not an MRI-derived estimate.',
                    'quality_score': None, 'result_type': 'synthetic_demonstration', 'created_at': now(), 'updated_at':
 now()})
        items.append({'_id': ObjectId(), 'sample_code': f'DEMO-{i+1:04}', 'owner_id': researcher['_id'],
            'age_at_scan_weeks': age, 'birth_age_weeks': round(min(age, rng.uniform(25, 40)), 4),
            'sex': rng.choice(['Female', 'Male']), 'has_dmri': dmri, 'has_fmri': fmri,
            'visibility': 'shared', 'status': 'completed' if analyses else 'registered', 'source': 'synthetic',
            'description': 'Synthetic coursework fixture. No real person or image.', 'files': [],
            'notes': [{'_id': ObjectId(), 'author_id': researcher['_id'], 'text': 'Synthetic demonstration note.',
                'category': 'research', 'created_at': now(), 'updated_at': now()}],
            'analyses': analyses, 'created_at': now(), 'updated_at': now(), 'version': 1})
    db.samples.insert_many(items)
    return {'samples': len(items), 'analyses': sum(len(x['analyses']) for x in items)}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--database', default='brainage_demo')
    args = parser.parse_args()
    if not args.database.startswith('brainage_demo'):
        raise SystemExit('Demo database name must start with brainage_demo.')
    app = create_app({'MONGO_DB': args.database})
    print(seed(app.extensions['mongo_db']))

if __name__ == '__main__':
    main()
```

19. scripts/verify_exports.py

SHA-256: aa7c41b03611895b6f6f5a4c77564234992331b96fc0f5d6d800db1c9105f3a5

```
"""Verify a synthetic mongoexport/mongoimport round trip in a disposable database."""
import json
import subprocess
import uuid
from bson import ObjectId
from datetime import datetime
from brainage import ROOT, create_app
from scripts.import_database import restore_export

def main():
    app = create_app()
    client = app.extensions['mongo_client']
    name = 'brainage_roundtrip_' + uuid.uuid4().hex
    target = client[name]
    try:
        counts = restore_export(client, app.config['MONGO_URI'], name, ROOT / 'exports/demo')
        assert counts['samples'] == 150
        sample = target.samples.find_one()
        assert isinstance(sample['_id'], ObjectId)
        assert isinstance(sample['created_at'], datetime)
        assert isinstance(sample['notes'][0]['_id'], ObjectId)
        assert target.samples.count_documents({'source': {'$ne': 'synthetic'}}) == 0
        report = {'status': 'passed', 'counts': counts,
            'objectid_and_datetime_types_preserved': True,
            'note': 'Actual mongoexport output imported into an isolated real MongoDB database.'}
        (ROOT / 'runtime/export-verification.json').write_text(json.dumps(report, indent=2), encoding='utf-8')
        print(json.dumps(report, indent=2))
    finally:
        assert name.startswith('brainage_roundtrip_')
        client.drop_database(name)

if __name__ == '__main__':
    main()
```

20. tests/conftest.py

SHA-256: 087bace2e8d3a97ad0f48d6a2d06d088a8d6e3dc4959247f5df624169977ca84

```
import uuid
import pytest
from bson import ObjectId
from brainage import create_app
from brainage.auth import hash_password
from brainage.common import now

@pytest.fixture(scope='session')
def app():
    name = 'brainage_test_' + uuid.uuid4().hex
    app = create_app({'TESTING': True, 'MONGO_DB': name, 'JWT_SECRET': 'test-only-key-' + 'x'*40})
    yield app
    assert name.startswith('brainage_test_')
    app.extensions['mongo_client'].drop_database(name)
    app.extensions['mongo_client'].close()

@pytest.fixture(scope='session')
def identities():
    return [{'_id': ObjectId(), 'username': name, 'display_name': name, 'role': role,
        'password_hash': hash_password('TestingPass!2026'), 'active': True, 'auth_version': 0, 'created_at': now()}
        for name, role in [('admin', 'admin'), ('alice', 'researcher'), ('bob', 'researcher')]]

@pytest.fixture(autouse=True)
def reset(app, identities):
    database = app.extensions['mongo_db']
    assert database.name.startswith('brainage_test_')
    for name in database.list_collection_names():
        database[name].delete_many({})
    database.users.insert_many([dict(x) for x in identities])

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def database(app):
    return app.extensions['mongo_db']

@pytest.fixture
def tokens(client):
    result = {}
    for name in ('admin', 'alice', 'bob'):
        response = client.post('/api/v1/auth/login', json={'username': name, 'password': 'TestingPass!2026'})
        assert response.status_code == 200
        result[name] = {'Authorization': 'Bearer ' + response.json['access_token']}
    return result

@pytest.fixture
def make_sample(client, tokens):
    def create(owner='alice', **fields):
        payload = {'sample_code': 'TEST-' + uuid.uuid4().hex[:8], 'age_at_scan_weeks': 40,
            'birth_age_weeks': 38, 'has_dmri': True, 'has_fmri': True, **fields}
        response = client.post('/api/v1/samples', json=payload, headers=tokens[owner])
        assert response.status_code == 201, response.json
        return response.json['data']
    return create
```

21. tests/test_api.py

SHA-256: e488b184b11ded2e529b3f6ef6556f1d45107589dfdaf34772a456f9f75bceb1

```
import pytest
import jwt
from datetime import timedelta
from bson import ObjectId
from flask import g
from brainage.common import APIError, now, update_sample

BASE = '/api/v1'
RESULT = {'modality': 'dmri', 'estimated_age_weeks': 41.5, 'method': 'Test fixture',
    'model_version': 'test-v1', 'provenance': 'Invented test data, not inference', 'quality_score': 0.8}

def test_health_and_errors(client):
    assert client.get(BASE+'/health').json['status'] == 'ok'
    assert client.get(BASE+'/missing').status_code == 404
    assert client.post(BASE+'/health').status_code == 405
    assert client.get(BASE+'/samples').status_code == 401
    assert client.get(BASE+'/samples', headers={'Authorization': 'Bearer fake'}).status_code == 401

def test_registration_approval_and_no_privilege_escalation(client, tokens):
    payload = {'username': 'new.user', 'password': 'NewPassword!2026', 'display_name': 'New User'}
    assert client.post(BASE+'/auth/register', json={**payload, 'role': 'admin'}).status_code == 400
    response = client.post(BASE+'/auth/register', json=payload)
    assert response.status_code == 201
    user_id = response.json['data']['_id']
    assert 'password_hash' not in response.json['data']
    assert client.post(BASE+'/auth/register', json=payload).status_code == 409
    credentials = {k: payload[k] for k in ('username', 'password')}
    assert client.post(BASE+'/auth/login', json=credentials).status_code == 403
    assert client.patch(BASE+'/users/'+user_id, json={'active': True}, headers=tokens['admin']).status_code == 200
    assert client.post(BASE+'/auth/login', json=credentials).status_code == 200

def test_auth_revocation_and_expiration(client, tokens, app):
    assert client.get(BASE+'/auth/me', headers=tokens['alice']).status_code == 200
    assert client.post(BASE+'/auth/logout', headers=tokens['alice']).status_code == 204
    assert client.get(BASE+'/auth/me', headers=tokens['alice']).status_code == 401
    claims = jwt.decode(tokens['bob']['Authorization'][7:], app.config['JWT_SECRET'], algorithms=['HS256'],
audience='brainage-api')
    claims['exp'] = now() - timedelta(seconds=1)
    expired = jwt.encode(claims, app.config['JWT_SECRET'], algorithm='HS256')
    assert client.get(BASE+'/auth/me', headers={'Authorization': 'Bearer '+expired}).status_code == 401

def test_password_change_revokes_sessions(client, tokens):
    response = client.put(BASE+'/auth/password', headers=tokens['alice'], json={
        'current_password': 'TestingPass!2026', 'new_password': 'ChangedPassword!2026'})
    assert response.status_code == 200
    assert client.get(BASE+'/auth/me', headers=tokens['alice']).status_code == 401
    assert client.post(BASE+'/auth/login', json={'username': 'alice', 'password': 'TestingPass!2026'}).status_code ==
401
    assert client.post(BASE+'/auth/login', json={'username': 'alice', 'password': 'ChangedPassword!2026'}).status_code
== 200

@pytest.mark.parametrize('payload', [[], None, {'username': 'ab', 'password': 'long-password'},
    {'username': 'alice', 'password': 'short'}, {'username': {'$ne': None}, 'password': 'TestingPass!2026'},
    {'username': 'alice', 'password': '界'*25}])
def test_auth_input_validation(client, payload):
    response = client.post(BASE+'/auth/login', json=payload)
    assert response.status_code in (400, 415)

def test_sample_lifecycle_and_partial_update(client, tokens, make_sample):
    item = make_sample()
    path = BASE+'/samples/'+item['_id']
    response = client.put(path, json={'description': 'Updated'}, headers=tokens['alice'])
    assert response.status_code == 200
    assert response.json['data']['age_at_scan_weeks'] == 40
    assert response.json['data']['version'] == 2
    assert client.get(path, headers=tokens['alice']).status_code == 200
    assert client.delete(path, headers=tokens['alice']).status_code == 204
    assert client.get(path, headers=tokens['alice']).status_code == 404

def test_sample_access_control(client, tokens, make_sample):
    item = make_sample()
    path = BASE+'/samples/'+item['_id']
    assert client.get(path, headers=tokens['bob']).status_code == 404
    assert client.get(BASE+'/samples', headers=tokens['bob']).json['pagination']['total'] == 0
    assert client.get(BASE+'/statistics/overview', headers=tokens['bob']).json['data']['total'] == 0
    assert client.patch(path, headers=tokens['alice'], json={'visibility': 'shared'}).status_code == 200
```

```
    assert client.get(path, headers=tokens['bob']).status_code == 200
    assert client.patch(path, headers=tokens['bob'], json={'status': 'failed'}).status_code == 403
    assert client.delete(path, headers=tokens['bob']).status_code == 403
    assert client.patch(path, headers=tokens['admin'], json={'status': 'ready'}).status_code == 200

@pytest.mark.parametrize('field,value', [('age_at_scan_weeks', True), ('age_at_scan_weeks', '40'),
    ('age_at_scan_weeks', 10), ('age_at_scan_weeks', 90), ('age_at_scan_weeks', float('nan')),
    ('birth_age_weeks', 44), ('has_dmri', 'true'), ('has_fmri', 1), ('visibility', 'public'),
    ('status', 'diagnosed'), ('sex', []), ('sample_code', {'$ne': None}), ('sample_code', 'a/b'),
    ('source', 'dhcp_local'), ('owner_id', 'x'), ('description', 'x'*2001)])
def test_sample_validation(client, tokens, field, value):
    response = client.post(BASE+'/samples', headers=tokens['alice'], json={
        'sample_code': 'TEST', 'age_at_scan_weeks': 40, field: value})
    assert response.status_code == 400, response.json

def test_duplicate_and_missing(client, tokens, make_sample):
    make_sample(sample_code='DUP')
    assert client.post(BASE+'/samples', headers=tokens['alice'], json={'sample_code': 'DUP', 'age_at_scan_weeks':
40}).status_code == 409
    assert client.get(BASE+'/samples/not-an-id', headers=tokens['alice']).status_code == 400
    assert client.get(BASE+'/samples/'+'0'*24, headers=tokens['alice']).status_code == 404
    assert client.post(BASE+'/samples', data='{}', headers=tokens['alice']).status_code == 415
    assert client.post(BASE+'/samples', data='{bad', content_type='application/json',
headers=tokens['alice']).status_code == 400

@pytest.mark.parametrize('query', ['page=0', 'page=abc', 'page_size=101', 'page_size=-1', 'sort=password_hash',
    'age_min=45&age_max=30', 'age_min=NaN', 'age_min=hello', 'modality=invalid', 'status=unknown',
    'unknown=yes', 'page=1&page=2', 'order=random', 'page=99999999999999999999999999999999999999'])
def test_query_validation(client, tokens, query):
    assert client.get(BASE+'/samples?'+query, headers=tokens['alice']).status_code == 400

def test_search_sort_filters_pagination(client, tokens, make_sample):
    for i in range(5):
        make_sample(sample_code=f'COHORT-{i}', age_at_scan_weeks=38+i)
    response =
client.get(BASE+'/samples?q=COHORT&age_min=39&age_max=42&modality=both&page_size=2&sort=age_at_scan_weeks&order=desc',
headers=tokens['alice'])
    assert response.json['pagination'] == {'page': 1, 'page_size': 2, 'pages': 2, 'total': 4}
    assert [x['age_at_scan_weeks'] for x in response.json['data']] == [42, 41]
    assert all('notes' not in x for x in response.json['data'])
    assert client.get(BASE+'/samples?q=.*', headers=tokens['alice']).json['pagination']['total'] == 0

@pytest.mark.parametrize('kind,payload', [('notes', {'text': 'A note', 'category': 'quality'}), ('analyses', RESULT)])
def test_nested_crud_owner_and_parent_scope(client, tokens, make_sample, kind, payload):
    item = make_sample(visibility='shared')
    path = BASE+'/samples/'+item['_id']+'/'+kind
    created = client.post(path, headers=tokens['bob'], json=payload)
    assert created.status_code == 201, created.json
    identifier = created.json['data']['_id']
    assert client.get(path, headers=tokens['alice']).json['pagination']['total'] == 1
    assert client.get(path+'/'+identifier, headers=tokens['alice']).status_code == 200
    update = {'text': 'Revised note'} if kind == 'notes' else {'estimated_age_weeks': 42}
    assert client.put(path+'/'+identifier, headers=tokens['alice'], json=update).status_code == 403
    revised = client.put(path+'/'+identifier, headers=tokens['bob'], json=update)
    assert revised.status_code == 200
    if kind == 'analyses':
        assert revised.json['data']['age_difference_weeks'] == 2
    other = make_sample(visibility='shared')
    assert client.get(BASE+'/samples/'+other['_id']+'/'+kind+'/'+identifier, headers=tokens['bob']).status_code == 404
    assert client.delete(path+'/'+identifier, headers=tokens['bob']).status_code == 204
    assert client.get(path+'/'+identifier, headers=tokens['alice']).status_code == 404

def test_analysis_validation_and_age_recalculation(client, tokens, make_sample):
    item = make_sample()
    path = BASE+'/samples/'+item['_id']
    for payload in [{**RESULT, 'quality_score': 2}, {**RESULT, 'estimated_age_weeks': True},
        {**RESULT, 'provenance': ''}, {**RESULT, 'age_difference_weeks': 99}]:
        assert client.post(path+'/analyses', headers=tokens['alice'], json=payload).status_code == 400
    assert client.post(path+'/analyses', headers=tokens['alice'], json=RESULT).status_code == 201
    assert client.patch(path, headers=tokens['alice'], json={'has_dmri': False}).status_code == 409
    response = client.patch(path, headers=tokens['alice'], json={'age_at_scan_weeks': 41})
    assert response.json['data']['analyses'][0]['age_difference_weeks'] == .5
    response = client.patch(path, headers=tokens['alice'], json={'age_at_scan_weeks': None})
    assert response.json['data']['analyses'][0]['age_difference_weeks'] is None
    missing = make_sample(has_dmri=False)
    assert client.post(BASE+'/samples/'+missing['_id']+'/analyses', headers=tokens['alice'], json=RESULT).status_code
== 400

def test_admin_permissions_and_immediate_deactivation(client, tokens):
    assert client.get(BASE+'/users', headers=tokens['alice']).status_code == 403
    users = client.get(BASE+'/users', headers=tokens['admin']).json['data']
```

```
    assert all('password_hash' not in user for user in users)
    bob = next(user for user in users if user['username'] == 'bob')
    admin = next(user for user in users if user['username'] == 'admin')
    assert client.put(BASE+'/users/'+admin['_id'], headers=tokens['admin'], json={'active': False}).status_code == 409
    assert client.put(BASE+'/users/'+bob['_id'], headers=tokens['admin'], json={'active': False}).status_code == 200
    assert client.get(BASE+'/auth/me', headers=tokens['bob']).status_code == 401
    assert client.get(BASE+'/audit-logs', headers=tokens['admin']).json['pagination']['total'] == 1
    assert client.get(BASE+'/audit-logs', headers=tokens['alice']).status_code == 403

def test_aggregation_latest_pairing_and_visibility(client, tokens, make_sample):
    item = make_sample()
    path = BASE+'/samples/'+item['_id']+'/analyses'
    for mode, estimate in [('dmri', 39), ('dmri', 42), ('fmri', 40)]:
        assert client.post(path, headers=tokens['alice'], json={**RESULT, 'modality': mode, 'estimated_age_weeks':
estimate}).status_code == 201
    rows = client.get(BASE+'/statistics/maturity', headers=tokens['alice']).json['data']
    assert next(x for x in rows if x['_id']['modality'] == 'dmri')['mean_estimated_age_weeks'] == 42
    pair = client.get(BASE+'/statistics/comparison', headers=tokens['alice']).json['data'][0]
    assert pair['paired_sample_count'] == 1 and pair['mean_dmri_minus_fmri_weeks'] == 2
    for endpoint in ('maturity', 'comparison', 'cohorts'):
        assert client.get(BASE+'/statistics/'+endpoint, headers=tokens['bob']).json['data'] == []
    cohort = client.get(BASE+'/statistics/cohorts', headers=tokens['alice']).json['data'][0]
    assert cohort['sample_count'] == 1 and cohort['_id']['age_group'] == '37-<42 weeks'
    csv = client.get(BASE+'/exports/cohorts?format=csv', headers=tokens['alice'])
    assert csv.status_code == 200 and 'text/csv' in csv.content_type and '37-<42 weeks' in csv.text

def test_imported_identity_protected(client, tokens, make_sample, database):
    item = make_sample()
    database.samples.update_one({'_id': ObjectId(item['_id'])}, {'$set': {'source': 'dhcp_local'}})
    assert client.patch(BASE+'/samples/'+item['_id'], headers=tokens['alice'], json={'sample_code': 'NEW'}).status_code
== 400

def test_optimistic_concurrency(app, database, identities, make_sample):
    item = make_sample()
    original = database.samples.find_one({'_id': ObjectId(item['_id'])})
    with app.test_request_context():
        g.user = identities[1]
        update_sample(original, {'description': 'first'}, 'test')
        with pytest.raises(APIError) as error:
            update_sample(original, {'description': 'stale'}, 'test')
        assert error.value.status == 409
    assert database.samples.find_one({'_id': original['_id']})['description'] == 'first'

def test_statistics_model_selection_and_age_groups(client, tokens, make_sample):
    for age in (31.9, 32, 37, 42, None):
        item = make_sample(age_at_scan_weeks=age, birth_age_weeks=None)
        path = BASE + '/samples/' + item['_id'] + '/analyses'
        for mode in ('dmri', 'fmri'):
            for method, version, estimate in (('A', 'v1', 40), ('B', 'v2', 45)):
                result = {**RESULT, 'modality': mode, 'method': method,
                          'model_version': version, 'estimated_age_weeks': estimate}
                assert client.post(path, headers=tokens['alice'], json=result).status_code == 201
    url = BASE + '/statistics/maturity?method=A&model_version=v1&group_by=age_group'
    rows = client.get(url, headers=tokens['alice']).json['data']
    dmri = [r for r in rows if r['_id']['modality'] == 'dmri']
    assert {r['_id']['age_group'] for r in dmri} == {'<32 weeks', '32-<37 weeks', '37-<42 weeks', '>=42 weeks',
'unknown'}
    assert all(r['sample_count'] == 1 and r['mean_estimated_age_weeks'] == 40 for r in dmri)
    assert next(r for r in dmri if r['_id']['age_group'] == 'unknown')['difference_count'] == 0
    query = '?dmri_method=A&dmri_model_version=v1&fmri_method=B&fmri_model_version=v2'
    pair = client.get(BASE + '/statistics/comparison' + query, headers=tokens['alice']).json['data'][0]
    assert pair['paired_sample_count'] == 5 and pair['mean_dmri_minus_fmri_weeks'] == -5
    assert client.get(url, headers=tokens['bob']).json['data'] == []
    assert client.get(BASE + '/statistics/maturity?group_by=invalid', headers=tokens['alice']).status_code == 400
```

22. static/app.js

SHA-256: dba7f58a7b2fe1bd93dc14e38a7af50acfc6456a2d5cf3692886194882b5ddc4

```
"use strict";
const $ = id => document.getElementById(id);
let token = null, page = 1, pages = 1, selected = null;
function message(text) { $("message").textContent = text; $("message").hidden = !text; }
async function api(path, options = {}) {
  const headers = {...(token ? {Authorization: "Bearer " + token} : {}), ...(options.body ? {"Content-Type":"application/json"} : {})};
  const result = await fetch("/api/v1" + path, {...options, headers});
  const data = result.status === 204 ? null : await result.json();
  if (!result.ok) {
    if (result.status === 401 && token) { token=null; $("workspace").hidden=true; $("login-panel").hidden=false; $("logout").hidden=true; $("detail").close(); }
    throw new Error(data?.error?.message || "Request failed");
  }
  return data;
}
function filters() { const q = new URLSearchParams(); for (const [k,v] of new FormData($("filters"))) if(v) q.set(k,v); return q; }
function cell(text) { const td=document.createElement("td"); td.textContent=text; return td; }
async function load() {
  const query=filters();
  const overview=await api("/statistics/overview?"+query);
  const synthetic=overview.data.by_source.length===1 && overview.data.by_source[0]._id==="synthetic";
  $("data-notice").textContent=synthetic ? "SYNTHETIC DEMO — All samples and estimates are invented test fixtures. No real images or clinical predictions." : "LOCAL RESEARCH DATA — File presence is not image quality control. Precomputed results require documented provenance.";
  $("dmri-label").textContent=synthetic?"Synthetic fixture flag":"Primary file presence";
  $("fmri-label").textContent=synthetic?"Synthetic fixture flag":"Primary file presence";
  $("paired-label").textContent=synthetic?"Synthetic paired fixtures":"Both local image types";
  const stats=overview.data.modalities[0] || {};
  $("total").textContent=overview.data.total; $("dmri").textContent=stats.dmri || 0; $("fmri").textContent=stats.fmri || 0; $("paired").textContent=stats.both || 0;
  query.set("page",page); query.set("page_size",10);
  const list=await api("/samples?"+query); pages=list.pagination.pages;
  $("rows").replaceChildren();
  for(const item of list.data){
    const row=document.createElement("tr");
    row.append(cell(item.sample_code),cell(item.age_at_scan_weeks == null ? "Unknown" : item.age_at_scan_weeks.toFixed(2)+" wk"),cell([item.has_dmri?"dMRI":"",item.has_fmri?"fMRI":""].filter(Boolean).join(" + ") || "Metadata only"),cell(item.status),cell(item.source));
    const td=cell(""); const button=document.createElement("button");button.textContent="View →";button.onclick=()=>openDetail(item._id).catch(e=>message(e.message));td.append(button);row.append(td);$("rows").append(row);
  }
  if(!list.data.length){const row=document.createElement("tr"), td=cell("No visible samples match these filters.");td.colSpan=6;row.append(td);$("rows").append(row);}
  $("page-info").textContent=`${list.pagination.total} sessions · Page ${page} of ${Math.max(1,pages)}`;
  $("previous").disabled=page<=1;$("next").disabled=page>=pages;
}
function entry(container,title,text){const box=document.createElement("div");box.className="entry";const label=document.createElement("strong"),p=document.createElement("p");label.textContent=title;p.textContent=text;box.append(label,p);container.append(box);}
async function openDetail(id){
  selected=id;const item=(await api("/samples/"+id)).data;
  $("detail-title").textContent=item.sample_code;
  $("detail-summary").textContent=`${item.source} · ${item.visibility} · ${item.files.length} indexed files · ${item.notes.length} notes`;
  $("notes").replaceChildren();$("analyses").replaceChildren();
  for(const note of item.notes)entry($("notes"),note.category,note.text);
  if(!item.notes.length)entry($("notes"),"No notes","Add the first research observation below.");
  for(const analysis of item.analyses)entry($("analyses"),`${analysis.modality} · ${analysis.estimated_age_weeks} weeks`,`${analysis.method} (${analysis.model_version})\nDifference: ${analysis.age_difference_weeks ?? "unknown"} weeks\n${analysis.provenance}`);
  if(!item.analyses.length)entry($("analyses"),"No precomputed results","No developmental age has been inferred for this sample.");
  $("raw").textContent=JSON.stringify(item,null,2);
  if(!$("detail").open)$("detail").showModal();
}
$("login-form").onsubmit=async event=>{event.preventDefault();message("");try{const data=await api("/auth/login",{method:"POST",body:JSON.stringify({username:$("username").value,password:$("password").value})});token=data.access_token;$("password").value="";$("account").textContent=data.user.display_name+" · "+data.user.role;$("login-panel").hidden=true;$("workspace").hidden=false;$("logout").hidden=false;page=1;await load();}catch(e){message(e.message);}};
$("filters").onsubmit=event=>{event.preventDefault();page=1;message("");load().catch(e=>message(e.message));};
$("previous").onclick=()=>{page--;load().catch(e=>message(e.message));};
$("next").onclick=()=>{page++;load().catch(e=>message(e.message));};
$("logout").onclick=async()=>{try{await api("/auth/logout",{method:"POST"});}finally{token=null;$("workspace").hidden=t
```

```
    rue;$("login-panel").hidden=false;$("logout").hidden=true;}};
57  $("close-detail").onclick=()=>$("detail").close();
58  $("note-form").onsubmit=async event=>{event.preventDefault();try{await api(`/samples/${selected}/notes`,{method:"POST",
    body:JSON.stringify({text:$("note-text").value,category:"research"})});$("note-text").value="";await
    openDetail(selected);}catch(e){message(e.message);$("detail").close();}};
```

23. static/index.html

SHA-256: eb040970571484d1b5bfdfbb6141797f9ec283f5e35c9eebdcf576ea411f43ec

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1">
<title>BrainAge Connect</title><link rel="stylesheet" href="/static/style.css"></head><body>
<aside><div class="brand"><span class="mark">B</span> BrainAge<span>Connect</span></div><p class="caption">RESEARCH
WORKSPACE</p><a class="active" href="#">◈   Sample explorer</a><a href="/api/v1/openapi.json" target="_blank">↗
  API schema</a><div class="aside-bottom">COM661 · Research data API<br><small>Local workspace /
v1.0</small></div></aside>
<main><header><div><p class="eyebrow">DEVELOPMENTAL NEUROIMAGING</p><h1>Sample explorer</h1><p class="muted">Manage
research records. Keep provenance in view.</p></div><button id="logout" class="secondary" hidden>Sign
out</button></header>
<div id="message" role="status" hidden></div>
<section id="login-panel" class="panel login"><div><span class="pill">LOCAL ACCESS</span><h2>Welcome to your
workspace</h2><p class="muted">Sign in to explore your permitted samples and analysis history.</p><p class="muted
small">Local administrator credentials are in <b>docs/LOCAL_ACCESS.txt</b>.<br>Demo accounts are listed in
README.md.</p></div><form id="login-form"><label>Username<input id="username" autocomplete="username"
required></label><label>Password<input id="password" type="password" autocomplete="current-password"
required></label><button>Sign in →</button></form></section>
<div id="workspace" hidden><p id="data-notice" class="footnote"></p><section class="metrics"><article><span>VISIBLE
SESSIONS</span><strong id="total">—</strong><small>Subject + scanning session</small></article><article><span>dMRI
AVAILABLE</span><strong id="dmri">—</strong><small id="dmri-label">Primary file
presence</small></article><article><span>fMRI AVAILABLE</span><strong id="fmri">—</strong><small
id="fmri-label">Primary file presence</small></article><article><span>PAIRED MODALITIES</span><strong
id="paired">—</strong><small id="paired-label">Both local image types</small></article></section>
<section class="panel"><div class="section-head"><h2>Research samples</h2><span id="account"
class="pill"></span></div><form id="filters" class="filters"><input name="q" placeholder="Search sample code…"
aria-label="Search sample code"><select name="modality" aria-label="Modality"><option value="">All
modalities</option><option value="both">Paired dMRI + fMRI</option><option value="dmri">dMRI</option><option
value="fmri">fMRI</option><option value="neither">Metadata only</option></select><input name="age_min" type="number"
step="any" min="20" max="80" placeholder="Min weeks" aria-label="Minimum scan age"><input name="age_max" type="number"
step="any" min="20" max="80" placeholder="Max weeks" aria-label="Maximum scan age"><button>Apply
filters</button></form>
<div class="table-wrap"><table><thead><tr><th>SAMPLE / SESSION</th><th>SCAN
AGE</th><th>MODALITIES</th><th>STATUS</th><th>SOURCE</th><th></th></tr></thead><tbody
id="rows"></tbody></table></div><footer><span id="page-info"></span><div><button id="previous" class="secondary">←
Previous</button><button id="next" class="secondary">Next →</button></div></footer></section>
<p class="footnote">Ages are postmenstrual weeks. File availability does not establish image quality. Stored estimates
are research records, not clinical diagnoses.</p></div>
<dialog id="detail"><div class="section-head"><h2 id="detail-title">Sample details</h2><button id="close-detail"
class="secondary">Close ✕</button></div><p id="detail-summary" class="muted"></p><div
class="detail-grid"><section><h3>Research notes</h3><div id="notes"></div><form id="note-form"><label>Add a research
note<textarea id="note-text" maxlength="4000" required placeholder="Record a research or data-quality
observation…"></textarea></label><button>Add note</button></form></section><section><h3>Analysis history</h3><div
id="analyses"></div><p class="muted small">Use the API or Postman collection to create/edit precomputed results and
perform the full resource lifecycle.</p></section></div><details><summary>Record metadata and file
references</summary><pre id="raw"></pre></details></dialog>
</main><script src="/static/app.js"></script></body></html>
```

24. static/style.css

SHA-256: 70121cfb4aa8a8ff6cb75e28b436d3f395cccd13bdfc8c4bd4b9f34d3acc8747

```
1 :root{--ink:#173e40;--muted:#6e8384;--teal:#137d73;--line:#dfe9e7;--paper:#f4f7f6}*{box-sizing:border-box}body{margin:0
;background:var(--paper);font:14px/1.6 "Segoe UI",sans-serif;color:var(--ink)}aside{position:fixed;width:235px;height:1
00vh;background:#143c3e;color:#eaf3ef;padding:32px 25px}.brand{font-size:19px;font-weight:700;line-height:1.2}.brand>sp
an:last-child{display:block;font-weight:400;margin-left:43px}.mark{float:left;background:#bce5ce;color:#184e48;padding:
8px 12px;border-radius:12px;margin-right:10px}.caption{font-size:10px;letter-spacing:2px;color:#9cb9b6;margin-top:66px}
aside a{display:block;text-decoration:none;color:#bad1cd;padding:12px;margin:8px -8px;border-radius:8px}aside a.active{
background:#285355;color:white}.aside-bottom{position:absolute;bottom:30px;color:#a8c6c1;font-size:12px}main{margin-lef
t:235px;padding:38px 46px;max-width:1600px}header{display:flex;justify-content:space-between;align-items:center;margin-
bottom:28px}.eyebrow{font-size:10px;letter-spacing:2px;color:var(--teal);font-weight:700}h1{font-size:34px;letter-spaci
ng:-1px;margin:5px 0}h2{font-size:19px;margin:0}h3{font-size:15px}.muted{color:var(--muted)}.small{font-size:12px}.pane
l{background:white;border:1px solid var(--line);border-radius:15px;padding:25px;box-shadow:0 6px 22px
#163e3a04}.login{display:grid;grid-template-columns:1.3fr
1fr;gap:60px;padding:40px;max-width:970px;margin-top:60px}.login
h2{margin-top:20px;font-size:26px}.pill{display:inline-block;padding:4px 10px;border-radius:30px;background:#e5f2ec;col
or:#32776c;font-size:10px;letter-spacing:.6px}label{display:block;font-size:12px;font-weight:600;margin:12px
0}input,select,textarea{font:inherit;border:1px solid #d5e1df;padding:10px 12px;border-radius:7px;background:white;colo
r:var(--ink);width:100%;outline:none}input:focus,textarea:focus{border-color:var(--teal);box-shadow:0 0 0 3px
#137d7312}button{border:0;background:var(--teal);color:white;padding:11px 17px;border-radius:7px;font:600 12px "Segoe U
I",sans-serif;cursor:pointer;white-space:nowrap}button:hover{filter:brightness(.94)}button:disabled{opacity:.4;cursor:d
efault}.secondary{background:white;color:var(--ink);border:1px solid
var(--line)}.metrics{display:grid;grid-template-columns:repeat(4,1fr);gap:18px;margin-bottom:25px}.metrics
article{border:1px solid var(--line);border-radius:12px;background:white;padding:20px 24px}.metrics
span{font-size:10px;letter-spacing:1px;color:var(--muted)}.metrics
strong{display:block;font-size:37px;font-weight:500;line-height:1.6}.metrics small{color:var(--muted);font-size:11px}.s
ection-head{display:flex;align-items:center;justify-content:space-between;gap:10px}.filters{display:grid;grid-template-
columns:2fr 1.5fr .8fr .8fr auto;gap:10px;margin:22px 0}.table-wrap{overflow:auto}table{width:100%;border-collapse:coll
apse;text-align:left;white-space:nowrap}th{font-size:10px;letter-spacing:.6px;color:var(--muted);font-weight:600;backgr
ound:#f7faf8}th,td{padding:15px 12px;border-bottom:1px solid
var(--line)}td{font-size:12px}td:first-child{font-weight:600}td button{padding:5px 12px;background:#edf5f1;color:var(--
teal)}footer{display:flex;justify-content:space-between;align-items:center;padding-top:20px;font-size:12px;color:var(--
muted)}footer button{margin-left:6px}.footnote{font-size:11px;color:var(--muted);margin-top:20px}#message{background:#f
ff3dd;color:#755122;border:1px solid #e9d4ad;padding:12px
18px;border-radius:8px;margin-bottom:20px}dialog{width:min(1050px,95vw);border:1px solid var(--line);border-radius:15px
;padding:30px;color:var(--ink)}dialog::backdrop{background:#12373970}.detail-grid{display:grid;grid-template-columns:1f
r 1fr;gap:35px}.entry{padding:13px;background:#f4f8f6;border-radius:8px;margin-bottom:10px}.entry p{margin:3px 0;white-
space:pre-wrap}textarea{height:100px}pre{white-space:pre-wrap;overflow-wrap:anywhere;background:#f3f6f4;padding:15px;ma
x-height:350px;overflow:auto;font-size:11px}summary{cursor:pointer;color:var(--muted);margin-top:20px}[hidden]{display:
none!important}@media(max-width:1050px){aside{width:180px;padding:25px
15px}main{margin-left:180px;padding:25px}.filters{grid-template-columns:1fr 1fr}.metrics{grid-template-columns:1fr 1fr}
}@media(max-width:700px){aside{display:none}main{margin:0;padding:18px}.login,.detail-grid{grid-template-columns:1fr;ga
p:20px}.login{padding:25px;margin-top:20px}.metrics{gap:10px}.metrics
article{padding:14px}.filters{grid-template-columns:1fr 1fr}h1{font-size:28px}}
```

25. setup.ps1

SHA-256: b147b346ff6772704d237b5884fcb402d18d76bfbacdaa208c5743f4a3ca15e6

```
param([string]$PythonExe = 'python')
$ErrorActionPreference = 'Stop'
Set-Location -LiteralPath $PSScriptRoot
if (-not (Test-Path -LiteralPath '.venv\Scripts\python.exe')) {
    & $PythonExe -m venv .venv
    if ($LASTEXITCODE -ne 0) { throw 'Install Python 3.12+ or provide -PythonExe with the full Python path.' }
}
& '.\.venv\Scripts\python.exe' -m pip install -r requirements-dev.txt
if ($LASTEXITCODE -ne 0) { throw 'Dependency installation failed. Check your package index/network settings.' }
Write-Host 'Dependencies ready. Start MongoDB on port 27018, then run python -m scripts.bootstrap.'
```

26. start.ps1

SHA-256: be9a4c4a07a7dee5908e0f6ffd688f8fffa892b05def51c95d6119119eb0fe34

```
param([switch]$Demo, [switch]$OpenBrowser)
$ErrorActionPreference = 'Stop'
Set-Location -LiteralPath $PSScriptRoot
$pythonExe = Join-Path $PSScriptRoot '.venv\Scripts\python.exe'
if (-not (Test-Path -LiteralPath $pythonExe)) { throw 'Run setup.ps1 first.' }
$port = if ($Demo) { '5001' } else { '5000' }
$mode = if ($Demo) { 'demo' } else { 'api' }
$apiUrl = 'http://127.0.0.1:' + $port
try {
    $health = Invoke-RestMethod -Uri ($apiUrl + '/api/v1/health') -TimeoutSec 2
    if ($health.application -eq 'BrainAge Connect') {
        Write-Host ('Already running: ' + $apiUrl)
        if ($OpenBrowser) { Start-Process $apiUrl }
        exit 0
    }
} catch { }
$runtimePath = Join-Path $PSScriptRoot 'runtime'
$databasePath = Join-Path $runtimePath 'mongodb'
New-Item -ItemType Directory -Force $databasePath | Out-Null
& $pythonExe -c @'
import sys
from pymongo import MongoClient
try:
    MongoClient('mongodb://127.0.0.1:27018',serverSelectionTimeoutMS=1000).admin.command('ping')
except Exception:
    sys.exit(1)
'@
if ($LASTEXITCODE -ne 0) {
    $mongoExe = Get-Command mongod -ErrorAction SilentlyContinue | Select-Object -ExpandProperty Source -First 1
    if (-not $mongoExe) { $mongoExe = Get-ChildItem -LiteralPath (Join-Path $PSScriptRoot 'tools') -Recurse -Filter mongod.exe | Select-Object -ExpandProperty FullName -First 1 }
    if (-not $mongoExe) { throw 'MongoDB server not found. Install it or place portable MongoDB under tools/.' }
    $logPath = Join-Path $runtimePath 'mongodb.log'
    $mongoArgs = '--bind_ip 127.0.0.1 --port 27018 --dbpath "{0}" --logpath "{1}" --logappend' -f $databasePath,$logPath
    $databaseProcess = Start-Process -FilePath $mongoExe -ArgumentList $mongoArgs -WindowStyle Hidden -PassThru
    @{Id=$databaseProcess.Id; Started=$databaseProcess.StartTime.ToUniversalTime().ToString('o'); Path=$mongoExe} | ConvertTo-Json | Set-Content (Join-Path $runtimePath 'mongodb-process.json')
    Start-Sleep -Seconds 3
}
$env:MONGO_DB = 'brainage_connect'
& $pythonExe -m scripts.bootstrap
if ($LASTEXITCODE -ne 0) { throw 'Database/configuration initialization failed.' }
if ($Demo) {
    $env:MONGO_DB = 'brainage_demo'
    & $pythonExe -c "from brainage import create_app; import sys; d=create_app().extensions['mongo_db']; sys.exit(0 if d.samples.count_documents({}) else 1)"
    if ($LASTEXITCODE -ne 0) {
        & $pythonExe -m scripts.seed_demo
        if ($LASTEXITCODE -ne 0) { throw 'Demo seed failed.' }
    }
}
$env:PORT = $port
$apiProcess = Start-Process -FilePath $pythonExe -ArgumentList 'app.py' -WorkingDirectory $PSScriptRoot -WindowStyle Hidden -PassThru -RedirectStandardOutput (Join-Path $runtimePath ($mode + '.stdout.log')) -RedirectStandardError (Join-Path $runtimePath ($mode + '.stderr.log'))
@{Id=$apiProcess.Id; Started=$apiProcess.StartTime.ToUniversalTime().ToString('o'); Path=$pythonExe} | ConvertTo-Json | Set-Content (Join-Path $runtimePath ($mode + '-process.json'))
$ready = $false
for ($attempt = 0; $attempt -lt 20; $attempt++) {
    Start-Sleep -Milliseconds 500
    try {
        $health = Invoke-RestMethod -Uri ($apiUrl + '/api/v1/health') -TimeoutSec 1
        if ($health.application -eq 'BrainAge Connect') { $ready = $true; break }
    } catch { }
}
if (-not $ready) { throw ('API failed to start. Check runtime/' + $mode + '.stderr.log') }
Write-Host ('Ready: ' + $apiUrl)
if ($Demo) { Write-Host 'Demo login: demo.admin / DemoAdmin!2026' }
else { Write-Host 'Login credentials: docs/LOCAL_ACCESS.txt' }
Write-Host 'Runs in the background. Use STOP_PROJECT.cmd to stop project services.'
if ($OpenBrowser) { Start-Process $apiUrl }
```

27. stop.ps1

SHA-256: 1d66bc3c9d774f042feeb3f9d74b821f283a050235bfba5b69856c3deee9b2bd

```
$ErrorActionPreference = 'Stop'
foreach ($mode in @('api','demo')) {
    $recordPath = Join-Path $PSScriptRoot ('runtime\' + $mode + '-process.json')
    if (Test-Path -LiteralPath $recordPath) {
        $record = Get-Content -LiteralPath $recordPath -Raw | ConvertFrom-Json
        $service = Get-Process -Id $record.Id -ErrorAction SilentlyContinue
        if ($service -and $service.Path -eq $record.Path -and $service.StartTime.ToUniversalTime().ToString('o') -eq $record.Started) {
            taskkill /PID $record.Id /T /F
            if ($LASTEXITCODE -ne 0) { throw ('Could not stop project ' + $mode + ' process.') }
        }
    }
}
$recordPath = Join-Path $PSScriptRoot 'runtime\mongodb-process.json'
if (Test-Path -LiteralPath $recordPath) {
    $record = Get-Content -LiteralPath $recordPath -Raw | ConvertFrom-Json
    $service = Get-Process -Id $record.Id -ErrorAction SilentlyContinue
    if ($service -and $service.Path -eq $record.Path -and $service.StartTime.ToUniversalTime().ToString('o') -eq $record.Started) {
        & (Join-Path $PSScriptRoot '.venv\Scripts\python.exe') -c @'
from pymongo import MongoClient
from pymongo.errors import AutoReconnect
try:
    MongoClient('mongodb://127.0.0.1:27018', serverSelectionTimeoutMS=2000).admin.command('shutdown')
except AutoReconnect:
    pass
'@
    }
}
Write-Host 'Tracked project services stopped.'
```

28. RUN_TESTS.cmd

SHA-256: 3357d2bba2cc56a57cfb634931689095db790b90ae13be40b506e4a9f8720ad7

```
1  @echo off
2  cd /d "%~dp0"
3  ".venv\Scripts\python.exe" -m pytest -q
4  pause
```

29. START_DEMO.cmd

SHA-256: c4ded93bacf74dca353931bf8d88c7d6ec1f262897776c1bd853fe982cfb072d

```
1 @echo off
2 cd /d "%~dp0"
3 powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0start.ps1" -Demo -OpenBrowser
4 pause
```

30. START_PROJECT.cmd

SHA-256: 565c32402d0a70d822d1e5da798fec251e0d77a27e2055135990253e21839f7c

```
1 @echo off
2 cd /d "%~dp0"
3 powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0start.ps1" -OpenBrowser
4 pause
```

31. STOP_PROJECT.cmd

SHA-256: f16dd2f8a66195e8b3d643f7d340705ae89cc53a3dc6116c9af7e7f8df1b2912

```
1 @echo off
2 cd /d "%~dp0"
3 powershell -NoProfile -ExecutionPolicy Bypass -File "%~dp0stop.ps1"
4 pause
```

32. requirements.txt

SHA-256: 12e659159157573fb9f02ceef413e03d342deb2bb241508776a888bbca7c789e

```
1 Flask==3.1.3
2 flask-cors==6.0.5
3 PyJWT==2.15.1
4 bcrypt==5.0.0
5 pymongo==4.19.0
6 python-dotenv==1.2.4
7 waitress==3.0.2
```

33. requirements-dev.txt

SHA-256: 2b5ecdbefedf36f510d58b6f0ae4b68c9e95193c748ad9b59115d646c1403cfd

```
1 -r requirements.txt
2 pytest==9.1.1
3 requests==2.34.2
```

34. .env.example

SHA-256: acc3f3cc6883a22d3f8da079218e14b5e61c2fad1168df5624a8c92fd71a65dd

```
MONGO_URI=mongodb://127.0.0.1:27018
MONGO_DB=brainage_connect
JWT_SECRET=replace-with-a-random-secret-at-least-32-characters
PORT=5000
CORS_ORIGINS=http://localhost:4200,http://127.0.0.1:4200
```

35. .gitignore

SHA-256: a488c667e5ad3011a0d21178bd5e510f991e0a6a83e0bb31f9ab20b7c39d5a05

```
.venv/
.env
__pycache__/
.pytest_cache/
runtime/
tools/
data/private/
exports/local/
submission/
docs/LOCAL_ACCESS.txt
*.log
```